

DynO: Dynamic Onloading of Deep Neural Networks from Cloud to Device

MARIO ALMEIDA*, STEFANOS LASKARIDIS*,
 STYLIANOS I. VENIERIS*, and ILIAS LEONTIADIS*, Samsung AI Center, Cambridge, UK
 NICHOLAS D. LANE, Samsung AI Center, Cambridge & University of Cambridge, UK

Recently, there has been an explosive growth of mobile and embedded applications using convolutional neural networks (CNNs). To alleviate their excessive computational demands, developers have traditionally resorted to cloud offloading, inducing high infrastructure costs and a strong dependence on networking conditions. On the other end, the emergence of powerful SoCs is gradually enabling on-device execution. Nonetheless, low- and mid-tier platforms still struggle to run state-of-the-art CNNs sufficiently. In this paper, we present DynO, a distributed inference framework that combines the best of both worlds to address several challenges, such as device heterogeneity, varying bandwidth and multi-objective requirements. Key components that enable this are its novel CNN-specific data packing method, which exploits the variability of precision needs in different parts of the CNN when onloading computation, and its novel scheduler that jointly tunes the partition point and transferred data precision at run time to adapt inference to its execution environment. Quantitative evaluation shows that DynO outperforms the current state-of-the-art, improving throughput by over an order of magnitude over device-only execution and up to 7.9× over competing CNN offloading systems, with up to 60× less data transferred.

ACM Reference Format:

Mario Almeida, Stefanos Laskaridis, Stylianos I. Venieris, Ilias Leontiadis, and Nicholas D. Lane. 2022. DynO: Dynamic Onloading of Deep Neural Networks from Cloud to Device. *ACM Trans. Embedd. Comput. Syst.* 1, 1, Article 1 (January 2022), 24 pages. <https://doi.org/10.1145/3510831>

1 INTRODUCTION

The unprecedented predictive power of convolutional neural networks (CNNs) has led to their ever-increasing usage on mobile and embedded devices, transforming their capabilities and, consequently, our lives. At the same time, the complexity of state-of-the-art CNNs is increasing exponentially [60]. While servers take advantage of powerful processors and accelerators [5, 8, 18, 30, 63], mobile devices struggle to cope. Hence, developers frequently resort to simpler or heavily compressed CNNs at the expense of accuracy [4, 56], with degraded impact on the user experience.

More recently, device vendors have started releasing System-on-Chips (SoCs) that host specialized units for CNN acceleration [24]. Although this can facilitate on-device processing, *developers still have to support the wide variety of devices* that exist in the wild [1, 59]. This includes older devices, old- and mid-tier smartphones, wearables, IoT modules, smart TVs or even household appliances.

*Indicates equal contribution.

Authors' addresses: Mario Almeida, mario.a@samsung.com; Stefanos Laskaridis, mail@stefanos.cc; Stylianos I. Venieris, s.venieris@samsung.com; Ilias Leontiadis, i.leontiadis@samsung.com, Samsung AI Center, Cambridge, UK; Nicholas D. Lane, Samsung AI Center, Cambridge & University of Cambridge, UK, nic.lane@samsung.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1539-9087/2022/1-ART1 \$15.00

<https://doi.org/10.1145/3510831>

DynO: 深度神经网络从云到设备的动态加载

MARIO ALMEIDA*, STEFANOS LASKARIDIS*, STYLIANOS I. VENIERIS*
和 ILIAS LEONTIADIS*, 英国剑桥三星人工智能中心 NICHOLAS D. LANE
, 英国剑桥三星人工智能中心

最近, 使用卷积神经网络 (CNN) 的移动和嵌入式应用出现了爆炸性增长。为了缓解过多的计算需求, 开发人员传统上求助于云卸载, 这会导致高昂的基础设施成本和对网络条件的强烈依赖。另一方面, 强大的 SoC 的出现正在逐渐实现设备上的执行。尽管如此, 中低层平台仍然难以充分运行最先进的 CNN。在本文中, 我们提出了 DynO, 这是一种分布式推理框架, 它结合了两个领域的优点, 可以解决设备异构性、变化的带宽和多目标要求等多项挑战。实现这一目标的关键组件是其新颖的 CNN 特定数据打包方法, 该方法在加载计算时利用 CNN 不同部分的精度需求的变化性, 以及新颖的调度程序, 该调度程序在运行时联合调整分区点和传输数据精度, 以使推理适应其执行环境。定量评估表明, DynO 的性能优于当前最先进的技术, 与仅设备执行相比, 吞吐量提高了一个数量级以上, 与竞争的 CNN 卸载系统相比, 吞吐量提高了 7.9 $\times$, 传输的数据量减少了 60 $\times$ 。

ACM参考格式:

马里奥·阿尔梅达、斯特凡诺斯·拉斯卡里迪斯、斯蒂利亚诺斯·I·韦尼里斯、伊利亚斯·莱昂蒂亚迪斯和尼古拉斯·D·莱恩。2022.DynO: 深度神经网络从云到设备的动态加载。ACM 翻译。嵌入。计算。系统。1, 1, 第 1 条 (2022 年 1 月), 24 页。https://doi.org/10.1145/3510831

1 简介

卷积神经网络 (CNN) 前所未有的预测能力导致其在移动和嵌入式设备上的使用不断增加, 改变了它们的功能, 从而改变了我们的生活。与此同时, 最先进的 CNN 的复杂性呈指数级增长 [60]。虽然服务器利用强大的处理器和加速器 [5,8,18,30,63], 但移动设备却难以应对。因此, 开发人员经常采用更简单或高度压缩的 CNN, 以牺牲准确性为代价 [4, 56], 从而降低了用户体验的影响。

最近, 设备供应商开始发布片上系统 (SoC), 其中托管用于 CNN 加速的专用单元 [24]。尽管这可以促进设备上的处理, 但开发人员仍然必须支持现有的各种设备 [1, 59]。这包括旧设备、旧中型智能手机、可穿戴设备、物联网模块、智能电视甚至家用电器。

*Indicates equal contribution.

Authors' addresses: Mario Almeida, mario.a@samsung.com; Stefanos Laskaridis, mail@stefanos.cc; Stylianos I. Venieris, s.venieris@samsung.com; Ilias Leontiadis, i.leontiadis@samsung.com, Samsung AI Center, Cambridge, UK; Nicholas D. Lane, Samsung AI Center, Cambridge & University of Cambridge, UK, nic.lane@samsung.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1539-9087/2022/1-ART1 \$15.00

https://doi.org/10.1145/3510831

Supporting such a heterogeneous device landscape while sustaining high performance with a single CNN model poses significant challenges [1, 3].

In this endeavor, CNN developers who seek state-of-the-art performance and wide device compatibility, typically rely on fully or partially offloading to a remote infrastructure, such as the cloud [22, 31]. With the advent of 5G Mobile Edge Computing (5G-MEC), offloading can even support AI applications that have very tight latency requirements such as VR/AR, UAVs and robotics [6]. While offloading can improve inference latency and resolve the problem of supporting heterogeneous devices, *it also results in high operating costs* [58]. Moreover, remote execution can also raise privacy concerns [39, 45, 53] and yield inconsistent user experience due to varying networking conditions [62].

Despite the downsides, the mobile machine learning (ML) landscape is currently dominated by cloud-based models [2]. Therefore, there is a growing interest from these ML service providers on how to provide the already existing cloud-based services, while reducing the operational cost and at the same time keeping the existing performance characteristics. Nonetheless, the wide majority of CNN offloading works fail to capture this multi-objective formulation, focusing on single *client-side* optimization objectives, such as client latency and throughput.

Inspired by these attempts, we aim to combine the best of both worlds: i) the cloud's elastic computational power and support of diverse devices and ii) the emergence of embedded chipsets with enhanced CNN processing capabilities, under a synergistic device-server setup. Contrary to most offloading works [31, 41], which focus on pushing as much on-device computation to the cloud as possible, we explore the concept of *onloading*; we allow server-based CNN applications to push as much computation as possible onto the embedded devices in order to exploit their growing computational power. Under this paradigm, the goal is to minimize the remote-end usage, and hence cost, while still meeting the application's service-level objectives (SLOs).

In this context, we propose DynO, a *distributed CNN inference* framework that splits the computation between the client device, where the data originally reside, and a more powerful remote end. DynO employs a novel online scheduler which partitions the computation in such a manner so as to meet the latency and throughput SLOs, while minimizing the cloud load and the associated cost by means of a device-onloading policy. To boost the attainable performance, we exploit our observation that *different parts of a CNN have varying numerical precision requirements* and introduce a novel packing technique that dynamically quantizes and compresses the data transferred at the computation split. Upon deployment, the system monitors the inference runtime and dynamically adapts the onloading policy (split-point selection and data packing) to the varying computational load and network conditions. Moreover, our system avoids the need for costly model modifications or retraining, by only quantizing the activations to be transferred. This way, a *train-once-deploy-everywhere* workflow can be adopted, eliminating the need for maintaining different models for each device tier. Notably, this paper makes the following key contributions:

- We propose DynO, an automated distributed CNN inference framework that *onloads* as much computation as possible from the cloud into the devices, in order to meet the applications' requirements. DynO dynamically adapts to changes in device capabilities and load as well as the networking conditions, achieving up to 7.9× higher throughput than the state-of-the-art.
- We empirically show that different parts of a CNN manifest varying tolerance to quantization. Exploiting this, we propose a novel data packing method for optimizing the device-server communication. As such, the data to be transferred upon onloading are quantized down to different precisions before transmission. To push to more compact representations, we devise a novel *input-dependent quantization* technique, ISQuant, that uses the dynamic range of each input

支持这种异构设备环境，同时使用单个 CNN 模型维持高性能提出了重大挑战 [1, 3]。

在这一努力中，寻求最先进性能和广泛设备兼容性的 CNN 开发人员通常依赖于完全或部分卸载到远程基础设施，例如云 [22, 31]。随着 5G 移动边缘计算 (5G-MEC) 的出现，卸载甚至可以支持延迟要求非常严格的 AI 应用程序，例如 VR/AR、无人机和机器人 [6]。虽然卸载可以改善推理延迟并解决支持异构设备的问题，但它也会导致较高的运营成本[58]。此外，远程执行还会引起隐私问题[39,45,53]，并由于不同的网络条件而产生不一致的用户体验[62]。

尽管存在这些缺点，移动机器学习 (ML) 领域目前仍以基于云的模型为主[2]。因此，这些机器学习服务提供商对如何提供现有的基于云的服务，同时降低运营成本并保持现有的性能特征越来越感兴趣。尽管如此，绝大多数 CNN 卸载工作未能捕获这种多目标公式，而是专注于单个客户端优化目标，例如客户端延迟和吞吐量。

受这些尝试的启发，我们的目标是结合两方面的优势：i) 云的弹性计算能力和对不同设备的支持，ii) 在协同设备-服务器设置下，具有增强 CNN 处理能力的嵌入式芯片组的出现。与大多数卸载工作[31, 41]相反，它们专注于将尽可能多的设备上计算推送到云端，我们探索了加载的概念；我们允许基于服务器的 CNN 应用程序将尽可能多的计算推送到嵌入式设备上，以利用其不断增长的计算能力。在此范例下，目标是最大限度地减少远程端的使用，从而降低成本，同时仍然满足应用程序的服务级别目标 (SLO)。

在这种情况下，我们提出了 DynO，一种分布式 CNN 推理框架，它将计算在数据最初所在的客户端设备和更强大的远程端之间分开。DynO 采用了一种新颖的在线调度程序，该调度程序以某种方式对计算进行分区，以满足延迟和吞吐量 SLO，同时通过设备加载策略最小化云负载和相关成本。为了提高可达到的性能，我们利用我们的观察结果，即 CNN 的不同部分具有不同的数值精度要求，并引入了一种新颖的打包技术，该技术可以动态量化和压缩在计算分割时传输的数据。部署后，系统会监控推理运行时间，并根据不同的计算负载和网络条件动态调整加载策略（分割点选择和数据打包）。此外，我们的系统通过仅量化要转移的激活，避免了昂贵的模型修改或重新训练的需要。这样，就可以采用一次训练、随处部署的工作流程，从而无需为每个设备层维护不同的模型。值得注意的是，本文做出了以下关键贡献：

- 我们提出了 DynO，一种自动化的分布式 CNN 推理框架，它将尽可能多的计算从云端加载到设备中，以满足应用程序的要求。DynO 动态适应设备功能和负载以及网络条件的变化，吞吐量比最先进的技术高出 7.9×。
- 我们凭经验表明，CNN 的不同部分表现出不同的量化容忍度。利用这一点，我们提出了一种新颖的数据打包方法来优化设备-服务器通信。因此，加载时要传输的数据在传输之前被量化为不同的精度。为了推动更紧凑的表示，我们设计了一种新颖的依赖于输入的量化技术 ISQuant，它使用每个输入的动态范围

Table 1. Comparison of Distributed Inference Systems

Work	Offloading Granularity	Communication Optimization	Scheduler	Decision Variable(s)	Objectives	Additional Training
Neurosurgeon [31]	Layer	X	Dynamic, SO, Exhaustive	Split point	Latency, Energy	-
JALAD [41]	Layer	Q: dynamic bandwidth	Static, SO, ILP	Split point, bandwidth	Latency	-
DADS [22]	Layer	X	Dynamic, SO, Heuristic	Split point	Latency, Throughput	-
MoDNN [42]	neurons	X	Static, SO, Heuristic	Neuron partition	Latency	-
DeepThings [65]	Cross-layer tile	X	Static, SO, Manual	Tile size, no. of layers	Latency, Throughput	-
MCDNN [17]	Model	X	Dynamic, MO, Heuristic	Model variant, cloud or device	Latency, Energy	-
Clio [23]	Layer	Dynamic width	Dynamic, SO, Exhaustive	Split point, cloud-model width	Latency	Yes
Eir [64]	Image patch	X	Dynamic, SO, Multi-server	Patch packing, server allocation	Latency	Yes
IONN [29]	Layer	X	Dynamic, SO, Heuristic	Split point	Latency	-
Edgent [40]	Layer	EE	Dynamic, SO, Exhaustive	Split point, model depth	Latency	Yes
SPINN [37]	Layer	Q: fixed (8-bit) + EE	Dynamic, MO, Exhaustive	Split point, EE-policy	Latency, Throughput, Accuracy	Yes
DynO	Layer	ISQuant+BitShuffling+ LZ4	Dynamic, MO, Exhaustive	Split point, bandwidth	Server cost, Latency, Throughput, Accuracy	-

*Q: Quantization, EE: early-exiting, SO: single objective, MO: multi-objective, ILP: Integer Linear Programming

sample to adapt the resolution of the number format. This enables efficient communication between the involved parties, without sacrificing accuracy or requiring expensive fine-tuning. This technique is integrated into a low-overhead communication optimizer that is plug-in compatible with existing distributed inference systems, delivering up to 60× reduction in transmitted data with less than 1 percentage point (pp) accuracy drop.

- We introduce a novel CNN-tailored scheduler that adapts to connectivity changes, device load and optimization targets. Contrary to existing work, it explores a new design space by dynamically tuning *both* the split point and quantization precision. Moreover, it supports hard and soft constraints, capturing the multi-objective requirements of real-world apps. At run time, the scheduler considers all valid candidate (split point, precision) pairs and dynamically selects the best for the application needs.

2 RELATED WORK

Recently, a growing body of work has focused on collaboratively utilizing cloud and local resources for CNN inference. Unlike generic offloading [7, 9, 13], the latter exploit the nature of CNN workloads, *e.g.* the deterministic execution graph, to optimize offloading. One of the most prominent works is Neurosurgeon [31], a framework that selects a single split point to offload CNNs from device to server, minimizing either latency or energy. However, the evaluated CNNs were simple sequential models and unless server load is considered, the offloading decisions tend to be polarized (*i.e.* *offload-nothing*, *offload-everything*), depending on networking conditions.

Hu *et al.* [22] introduced a scheduling scheme to optimize either latency or throughput, based on the server load. However, the proposed scheduler only accounts for one objective at a time and lacks support for SLO deadlines. Closer to our work, JALAD [41] explored the latency-accuracy trade-off to make an offloading decision. Nonetheless, it requires excessive accuracy drop (up to 10%) in order to obtain meaningful performance gains, which cannot be tolerated in real-world apps. Furthermore, JALAD’s scheduler is based on a computationally expensive integer linear programming (ILP) solver which needs to run offline. Hence, as the scheduler cannot execute efficiently on resource-constrained devices, JALAD provides only static configurations and cannot adapt at run time to highly dynamic environments, such as mobile networks. Moreover, it ignores the device and server loads by solely considering the network variability, resulting in polarized decisions. Simultaneously, SPINN [37] and Clio [23] tackle offloading in a progressive manner focusing either on the depth [37] or width of the CNN [23]. Nonetheless, the flexibility of the progressive approach is also associated with a need for training, either for the early classifiers or the slicing-aware scheme, respectively. As such, there is an additional computational cost when targeting pretrained models. Instead, by not requiring any training phase, DynO can directly target any pretrained CNN, at no additional cost.

表 1. 分布式推理系统的比较

Work	Offloading Granularity	Communication Optimization	Scheduler	Decision Variable(s)	Objectives	Additional Training
Neurosurgeon [31]	Layer	$\times$	Dynamic, SO, Exhaustive	Split point	Latency, Energy	-
JALAD [41]	Layer	Q: dynamic bandwidth	Static, SO, ILP	Split point, bandwidth	Latency	-
DADS [22]	Layer	$\times$	Dynamic, SO, Heuristic	Split point	Latency, Throughput	-
MoDNN [42]	neurons	$\times$	Static, SO, Heuristic	Neuron partition	Latency	-
DeepThings [65]	Cross-layer tile	$\times$	Static, SO, Manual	Tile size, no. of layers	Latency, Throughput	-
MCDNN [17]	Model	$\times$	Dynamic, MO, Heuristic	Model variant, cloud or device	Latency, Energy	-
Clio [23]	Layer	Dynamic width	Dynamic, SO, Exhaustive	Split point, cloud-model width	Latency	Yes
Elr [64]	Image patch	$\times$	Dynamic, SO, Multi-server	Patch packing, server allocation	Latency	Yes
IONN [29]	Layer	$\times$	Dynamic, SO, Heuristic	Split point	Latency	-
Edgent [40]	Layer	EE	Dynamic, SO, Exhaustive	Split point, model depth	Latency, Throughput, Accuracy	Yes
SPINN [37]	Layer	Q: fixed (8-bit) + EE	Dynamic, MO, Exhaustive	Split point, EE-policy	Latency, Throughput, Accuracy	Yes

DynO Layer ISQuant+BitShuffling+ LZ4 动态、MO、详尽分割点、位宽 服务器成本、延迟、吞吐量、准确性 - *Q: 量化、EE: 提前退出、SO: 单目标、MO: 多目标、ILP: 整数线性规划

示例以适应数字格式的分辨率。这使得相关各方之间能够进行有效的沟通，而不会牺牲准确性或需要昂贵的微调。该技术集成到低开销通信优化器中，该优化器与现有分布式推理系统兼容插件，可将传输数据减少高达 60x，且精度下降不到 1 个百分点 (pp)。

- 我们引入了一种新颖的 CNN 定制调度程序，可以适应连接变化、设备负载和优化目标。与现有工作相反，它通过动态调整分割点和量化精度来探索新的设计空间。此外，它支持硬约束和软约束，捕获现实应用程序的多目标需求。在运行时，调度程序会考虑所有有效的候选(分割点、精度)对，并动态选择最适合应用程序需求的选项。

2 相关工作

最近，越来越多的工作集中在协作利用云和本地资源进行 CNN 推理。与通用卸载 [7,9,13] 不同，后者利用了 CNN 工作负载的性质，例如确定性执行图，以优化卸载。其中最著名的作品之一是 Neurosurgeon [31]，该框架选择单个分割点将 CNN 从设备卸载到服务器，从而最大限度地减少延迟或能耗。然而，评估的 CNN 是简单的顺序模型，除非考虑服务器负载，否则卸载决策往往会两极分化（即不卸载任何内容、卸载所有内容），具体取决于网络条件。

胡等人。[22]引入了一种调度方案，根据服务器负载来优化延迟或吞吐量。然而，提议的调度程序一次仅考虑一个目标，并且缺乏对 SLO 截止日期的支持。与我们的工作更接近的是，JALAD [41] 探索了延迟精度权衡以做出卸载决策。尽管如此，它需要过度的准确度下降（高达 10%）才能获得有意义的性能提升，这在现实世界的应用程序中是无法容忍的。此外，JALAD 的调度程序基于计算成本昂贵的整数线性规划 (ILP) 求解器，需要离线运行。因此，由于调度程序无法在资源受限的设备上有效执行，JALAD 仅提供静态配置，并且无法在运行时适应高度动态的环境，例如移动网络。此外，它仅考虑网络可变性而忽略了设备和服务器负载，从而导致两极分化的决策。同时，SPINN [37] 和 Clio [23] 以渐进的方式解决卸载问题，重点关注 CNN [23] 的深度 [37] 或宽度。尽管如此，渐进方法的灵活性也与训练的需要相关，无论是早期分类器还是切片感知方案。因此，针对预训练模型时会产生额外的计算成本。相反，由于不需要任何训练阶段，DynO 可以直接针对任何预训练的 CNN，无需额外成本。

With respect to the compression of the transferred data, SPINN also integrates a quantisation scheme, similar to DynO’s approach. However, contrary to DynO, the quantization level (*i.e.* the bitwidth) remains fixed at 8 bits and is uniform across all candidate splitting layers, while the scaling factors are statically selected and do not take into consideration the resilience of each layer to quantization, the actual dynamic range of the data or the instantaneous quality of connectivity. As a result, SPINN’s compression approach constitutes a subset of DynO’s compression method. DynO provides increased flexibility and communication efficiency through its broader space for compression, which allows different bitwidths across the candidate splitting layers, and provides higher adaptability by selecting the appropriate combination of bitwidth-split point based on the performance targets and networking conditions.

A different set of related work concerns offloading CNN inference to devices in the local network [42, 43, 65], to multiple servers for load balancing of denser tasks [64], to third-party edge devices along with the CNN computation graph [29], or to either device or cloud through model selection [17]. While many of the problems posed are closely related, these systems tend to have different requirements (*e.g.* multiple devices in local area network), optimization goals (*e.g.* scatter computation in a share-nothing setup) or significant overhead (*e.g.* maintenance of multiple models).

Table 1 presents a comparison of existing distributed inference systems. Compared to previous work, DynO introduces key differentiating factors for device-server synergistic inference. Specifically, our system scheduler actively monitors the CNN execution and adapts to the varying conditions, *e.g.* device load or networking. Our system leverages the reduced-precision resilience of the intermediate data to significantly compress the transmitted dependencies. Hence, the split point is no longer selected naively based only on the CNN architecture, but efficient data packing is also considered towards this decision. Thus, more split points are now attainable, since the amount of transferred data is not prohibitively high.

From a deployment aspect, DynO does not require any additional backend or model modification. Through a dynamic hooking mechanism, we maintain a model-agnostic approach that supports different architectures, *e.g.* multi-branch [57], residual [20] or depthwise-separable blocks [54]. This is especially important, as it allows for a plug-in runtime, without imposing retraining and model maintenance overhead to the user. Last, DynO incorporates an SLO-aware scheduler that shapes the onloading policy while minimizing cloud costs.

With respect to DynO’s input-dependent quantization technique, ISQuant, presented in Section 3.3, a similar approach has been recently added in the PyTorch deep learning framework under the name *dynamic quantization* [50]. Although the input-dependent derivation of the scale factor resembles ISQuant, there are crucial differences to our approach, namely: *i)* all layers are quantized, *ii)* the bitwidth is uniform across all layers, which can lead to degradation in accuracy, and *iii)* only 8-bit precision is supported. Instead, DynO quantizes only the activations of the selected splitting layer, supports any bitwidth and the bitwidths are nonuniform across candidate splitting layers, *i.e.* if a different layer is selected as a split point at run time, the bitwidth for quantizing the transferred data is also allowed to change as needed. This approach opens a whole new important dimension in determining the optimal split point, while having minimal impact on accuracy by isolating the quantization process to the data exchanged at the split point. In general, although adaptive quantization methods are slowly finding their way to commercialization, they are still quite coarse and inflexible, leading to potentially degraded accuracy and lack of deployability.

Another line of work that is tangential to ours is adaptive inference systems [36]. This class of systems aims to exploit the variability in the difficulty of different input samples in order to adapt the amount of computation based on the hardness of each sample. Such works span from dynamic DNNs that adapt the depth of the model [27, 34, 38, 49], apply dynamic channel pruning [26] or introduce progressive inference schemes for generative adversarial network (GAN)-based image

关于传输数据的压缩，SPINN 还集成了量化方案，类似于 DynO 的方法。然而，与 DynO 不同的是，量化级别（即位宽）保持固定为 8 位，并且在所有候选分裂层上都是统一的，而缩放因子是静态选择的，没有考虑每层的量化弹性、数据的实际动态范围或瞬时连接质量。因此，SPINN 的压缩方法构成了 DynO 压缩方法的子集。DynO 通过更广泛的压缩空间提供更高的灵活性和通信效率，允许跨候选分割层使用不同的位宽，并通过根据性能目标和网络条件选择适当的位宽分割点组合来提供更高的适应性。

一组不同的相关工作涉及将 CNN 推理卸载到本地网络中的设备 [42,43,65]，卸载到多个服务器以实现更密集任务的负载平衡 [64]，卸载到第三方边缘设备以及 CNN 计算图 [29]，或者通过模型选择卸载到设备或云 [17]。虽然提出的许多问题密切相关，但这些系统往往有不同的要求（例如局域网中的多个设备）、优化目标（例如无共享设置中的分散计算）或显着的开销（例如多个模型的维护）。

表 1 展示了现有分布式推理系统的比较。与之前的工作相比，DynO 引入了设备-服务器协同推理的关键差异化因素。具体来说，我们的系统调度程序主动监控 CNN 的执行并适应不同的条件，例如设备负载或网络。我们的系统利用中间数据的精度降低的弹性来显着压缩传输的依赖关系。因此，分割点不再仅根据 CNN 架构天真地选择，而是在这一决策中还考虑了高效的数据打包。因此，现在可以获得更多的分割点，因为传输的数据量并不算高得令人望而却步。

从部署方面来看，DynO 不需要任何额外的后端或模型修改。通过动态挂钩机制，我们维护了一种与模型无关的方法，支持不同的架构，例如多分支[57]、残差[20]或深度可分离块[54]。这一点尤其重要，因为它允许插件运行时，而不会给用户带来重新训练和模型维护的开销。最后，DynO 结合了一个 SLO 感知调度程序，可以制定加载策略，同时最大限度地降低云成本。

关于第 3.3 节中介绍的 DynO 的输入相关量化技术 ISQuant，最近在 PyTorch 深度学习框架中添加了一种类似的方法，名为动态量化 [50]。虽然比例因子的输入相关推导类似于 ISQuant，但我们的方法存在重大差异，即：i) 所有层都被量化，ii) 所有层的位宽都是统一的，这可能导致精度下降，iii) 仅支持 8 位精度。相反，DynO 仅量化所选分割层的激活，支持任何位宽，并且候选分割层之间的位宽不均匀，即，如果在运行时选择不同的层作为分割点，则还允许根据需要更改用于量化传输数据的位宽。这种方法在确定最佳分割点方面开辟了一个全新的重要维度，同时通过将量化过程与分割点交换的数据隔离，对精度的影响最小。总的来说，尽管自适应量化方法正在慢慢找到商业化的道路，但它们仍然相当粗糙和不灵活，导致潜在的准确性下降和缺乏可部署性。

与我们的工作无关的另一项工作是自适应推理系统[36]。此类系统旨在利用不同输入样本难度的变化，以便根据每个样本的难度调整计算量。此类工作涵盖动态 DNN，适应模型的深度 [27,34,38,49]，应用动态通道修剪 [26] 或为基于生成对抗网络 (GAN) 的图像引入渐进式推理方案

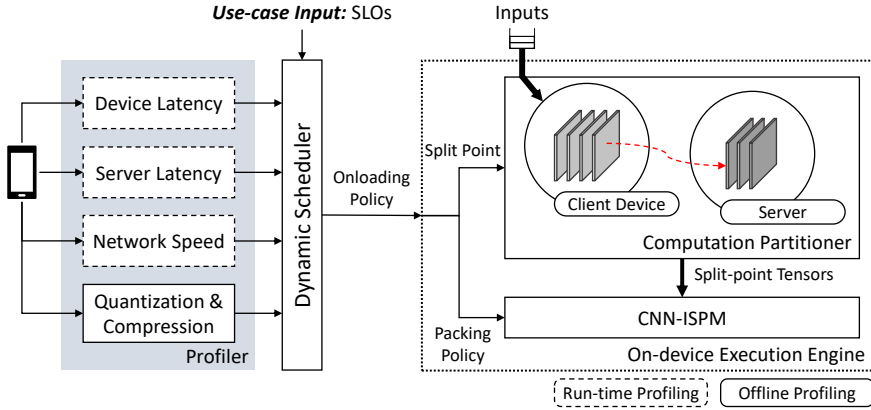


Fig. 1. DynO's system architecture.

generation [28]. In general, despite having different objectives, these approaches are related to DynO with respect to the fact that they provide the flexibility to tune the accuracy-efficiency trade-off of the end system.

3 DYNO

As aforementioned, the objective is to overcome the limitations of on-device or cloud execution in targeting heterogeneous devices while adapting to dynamic changes and application SLOs. To this end, we propose DynO (Fig. 1), a framework that addresses these challenges at three levels. First, from an execution perspective, a *distributed synergistic approach* is introduced with the device and cloud collaborating to run CNN inference. The CNN partitioning is parametrized to tunably assign computations to each end at run time, without modifications to the original model (§ 3.1, 3.2). Second, to minimize the transmission overhead and boost performance, a novel *CNN data packing method* (CNN-ISPM) is employed that compresses the transferred data beyond what was previously possible and with minimal impact on accuracy (§ 3.3). Finally, to adapt to dynamic changes, a *multi-objective scheduler* is developed which jointly determines the highest performing partitioning and data packing policy in order to meet the target SLOs (§ 3.4, 3.5).

3.1 Hooking and Instrumentation

To migrate computation between machines and support off-the-shelf models, DynO needs to be able to intercept and even modify CNN operations on-the-fly, transparently to the user. Typically, layers are represented by *modules* and data as multi-dimensional matrices, called *tensors*. DynO – implemented on top of PyTorch – intercepts and distributes computation *at module (layer) granularity*. To achieve this, we implemented a custom hooking framework that targets the PyTorch's base *module* class, replacing its call function with our own wrapper function. Fig. 2 shows an example of DynO's distributed inference. The layer numbers follow the static execution order observed during inference. Overall, DynO's wrapper function performs the following tasks:

Normal execution: During normal execution, the wrapper invokes the original module function with the original parameters, e.g. blocks 1 – 3 will be executed normally on the client. Furthermore, the wrapper collects runtime information for each executed block (see § 3.4).

Skipped execution: When an operation is assigned to a remote device the original PyTorch module is skipped. As a result, offloaded modules are not executed on the local device, e.g. blocks 4 – 6

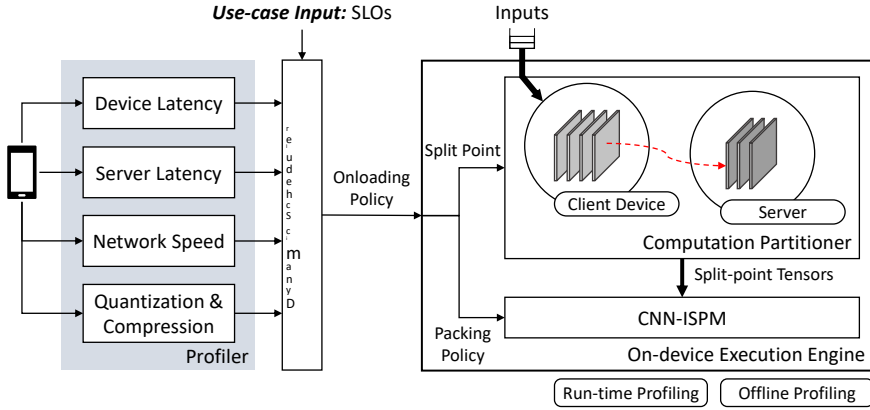


图 1. DynO 的系统架构。

一代[28]。一般来说，尽管目标不同，但这些方法都与 DynO 相关，因为它们提供了调整终端系统的准确性与效率权衡的灵活性。

3 动力

如上所述，目标是克服设备上或云执行在针对异构设备方面的限制，同时适应动态变化和应用程序 SLO。为此，我们提出了 DynO（图 1），这是一个在三个层面解决这些挑战的框架。首先，从执行的角度来看，引入了分布式协同方法，设备和云协作运行 CNN 推理。CNN 分区经过参数化，可以在运行时将计算可调地分配给每一端，而无需修改原始模型（第 3.1、3.2 节）。其次，为了最大限度地减少传输开销并提高性能，采用了一种新颖的 CNN 数据打包方法 (CNN-ISPM)，该方法将传输的数据压缩到超出以前的水平，并且对准确性的影响最小（第 3.3 节）。最后，为了适应动态变化，开发了一个多目标调度程序，它共同确定最高性能的分区和数据打包策略，以满足目标 SLO（第 3.4、3.5 节）。

3.1 挂钩和仪表

为了在机器之间迁移计算并支持现成模型，DynO 需要能够拦截甚至即时修改 CNN 操作，并且对用户透明。通常，层由模块和数据表示为多维矩阵，称为张量。DynO – 在 PyTorch 之上实现 – 以模块（层）粒度拦截和分发计算。为了实现这一目标，我们实现了一个针对 PyTorch 基模块类的自定义挂钩框架，用我们自己的包装函数替换其调用函数。图 2 显示了 DynO 分布式推理的示例。层编号遵循推理期间观察到的静态执行顺序。总的来说，DynO 的包装函数执行以下任务：

正常执行：在正常执行期间，包装器使用原始参数调用原始模块函数，例如块 1 – 3 将在客户端正常执行。此外，包装器收集每个执行块的运行时信息（参见第 3.4 节）。

跳过执行：当将操作分配给远程设备时，将跳过原始 PyTorch 模块。因此，卸载的模块不会在本地设备上执行，例如区块 4 – 6

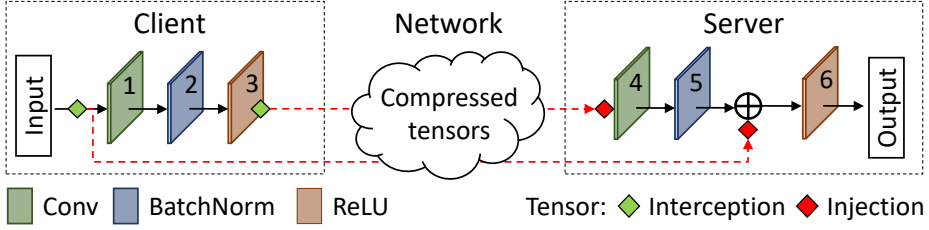


Fig. 2. DynO onloading part of a ResNet block.

will be skipped on the client.

Transfer execution: When computation needs to be transferred, the corresponding dependencies (i.e. tensors) are passed to the data packing queue (§ 3.3), e.g. the output of ops *input*¹ and 3 will be intercepted as soon as they are computed in order to be transferred to the server. DynO supports multiple transfers of control, in both directions.

Resume execution: When computation is to be resumed on the server, the dependencies are injected from the unpacking queue into the model and execution proceeds normally, e.g. the first dependency is injected before op 4, and the second just before addition.

3.2 Computation Partitioning

DynO can split a CNN in multiple arbitrary partitions and assign those to different devices to run. However, for the cloud onloading scenario, it rarely makes sense to have more than one partition in the real world due to the extra transmission cost which would dominate the overall latency. In essence, packing and transfer are overheads to the inference process, paid only to be later compensated by the faster runtime of the remote end. Once data reside there, we want to run as much as possible on the faster device. This is also evident in Fig. 9, where were DynO to use multiple points, we would pay multiple (un)packings and transfers to hop from the client to server and vice versa. Therefore, *we only consider one split point per inference*. Once the CNN is partitioned, we run one part on-device and the other on the cloud. This way we initially take advantage of data locality by starting inference on device and, subsequently, find a split point that minimizes the communication overhead if the user's device is not powerful enough to meet the execution latency requirements.

Internally, DynO captures the CNN workload as a directed acyclic graph (DAG), the *dependency graph*, with modules as vertices and data dependencies as edges. We deterministically assign a sequence id to the modules of the network based on the the topology of the DAG and the order of execution during the dependency graph construction phase. Finally, when partitioning a CNN with parallel branches, e.g. Inception-v3, or skip connections, e.g. ResNet, the dependencies (edges that cross the cut) have to be transmitted. These dependencies are traced in the offline profiling phase (§ 3.4) for every possible split in a single pass from the dependency graph along with other latency measurements. The selection scheme of the actual split point is detailed in § 3.5. In the example of Fig. 2, the nodes of the graph have already been indexed and the *input* tensor will be noted as a dependency when partitioning at node 3.

¹Note that the dependency on the output of the *input* operation represents a residual connection as used in ResNet networks. By no means does DynO send the input activations by default.

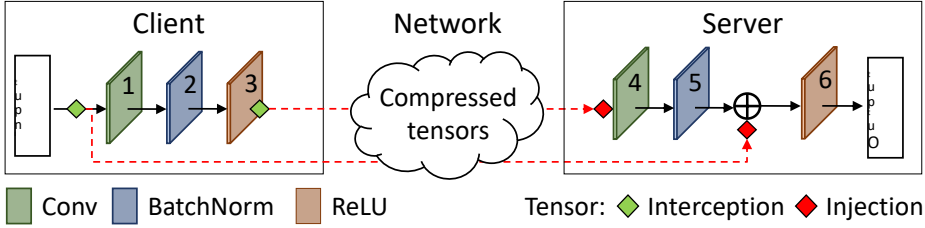


图 2. DynO 加载 ResNet 块的部分。

将在客户端上跳过。

传输执行：当计算需要传输时，相应的依赖项（即张量）被传递到数据打包队列 (§ 3.3)，例如ops input¹ 和 3 的输出将在计算后立即被拦截，以便传输到服务器。DynO 支持双向多重控制传输。

恢复执行：当服务器要恢复计算时，依赖项会从解包队列注入到模型中，并正常执行，例如第一个依赖项在操作 4 之前注入，第二个依赖项在添加之前注入。

3.2 计算分区

DynO 可以将 CNN 分割为多个任意分区，并将它们分配给不同的设备来运行。然而，对于云加载场景，在现实世界中拥有多个分区几乎没有意义，因为额外的传输成本将主导整体延迟。本质上，打包和传输是推理过程的开销，只是稍后通过远程端更快的运行时间来补偿。一旦数据驻留在那里，我们希望尽可能在更快的设备上运行。这在图 9 中也很明显，如果 DynO 使用多个点，我们将支付多次（拆包）包装和传输费用以从客户端跳到服务器，反之亦然。因此，我们每次推理只考虑一个分割点。CNN 分区后，我们将一部分在设备上运行，另一部分在云端运行。通过这种方式，我们首先通过在设备上启动推理来利用数据局部性，然后找到一个分割点，如果用户的设备不够强大，无法满足执行延迟要求，则可以最大限度地减少通信开销。

在内部，DynO 将 CNN 工作负载捕获为有向无环图 (DAG)（依赖图），其中模块作为顶点，数据依赖项作为边。我们根据 DAG 的拓扑结构和依赖图构建阶段的执行顺序确定性地为网络模块分配一个序列 id。最后，当用并行分支划分 CNN 时，例如 Inception-v3，或跳过连接，例如 ResNet，依赖关系（交叉切割的边）必须被传输。这些依赖关系在离线分析阶段（第 3.4 节）中针对依赖关系图的单次传递中的每个可能的分割以及其他延迟测量进行跟踪。实际分割点的选择方案详见 § 3.5。在图 2 的示例中，图的节点已被索引，并且在节点 3 处进行分区时，输入张量将被标记为依赖项。

¹Note that the dependency on the output of the *input* operation represents a residual connection as used in ResNet networks. By no means does DynO send the input activations by default.

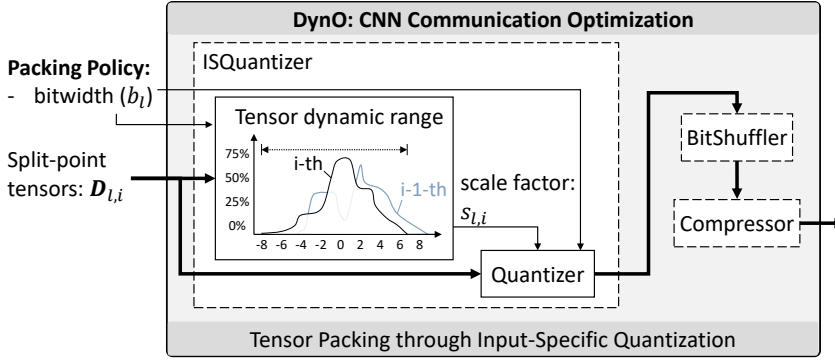


Fig. 3. CNN communication optimizer, CNN-ISPM.

3.3 CNN Communication Optimization

The size of the tensors manipulated by CNN layers can vary greatly, with many reaching several hundreds of KB even for single-input batches. Transferring these tensors can quickly outweigh the benefits of onloading, especially under poor network conditions. To overcome this and optimize the communication between the two processing parties, we introduce an input-specific packing module (CNN-ISPM). The key idea behind CNN-ISPM is the observation that 1) the output tensors of different layers require different degrees of precision for a given level of accuracy, *i.e.* each layer can have a different bitwidth, and 2) the data representation used at any given point in time needs to accommodate *only* the tensor values to be transferred to the remote end, *i.e.* the tensors that constitute dependencies of the selected split point, and further only their values for the *specific* input at hand. CNN-ISPM employs a CNN-specialized *three-phase packing* mechanism (Fig. 3) consisting of: input-specific precision quantization (ISQuant), bit shuffling and lossless compression.

Input-Specific Quantization: Precision quantization is frequently used in CNNs to reduce memory footprint and latency [47]. In terms of *which data* to quantize, most existing schemes target either both weights and activations [15] or only the weights [66] of all layers. In terms of *how* to quantize, existing works typically adopt linear quantization based on block floating-point (also known as dynamic fixed-point) [14, 15, 25, 33, 56] which dictates a uniform bitwidth and a non-uniform scale factor across the model's layers. This approach can be expressed as $q_l = \langle b, s_l \rangle$ for the l -th layer with b the bitwidth and s_l the scale factor that determines which bits to keep.

In the majority of existing works, the value of s_l is determined at design time by estimating the dynamic range of data over a calibration set (*e.g.* a subset of the target dataset). This approach is bounded by two main factors. First, a long profiling stage is required in order to determine the scale factors, with the strong assumption that the calibration set is representative of the inference-time data. Second, this approach underutilizes the representational range of the selected bitwidth in cases where the estimated dynamic range does not capture the current input's range. Fig. 4 illustrates such a case. The symbol $D_{l,i,k}$ denotes the k -th dependency tensor for the i -th input at split point l . The estimated dynamic range (grey dotted line) – that has been used at design time to set the scale factor to a fixed value – does not capture the actual range of the input tensor $D_{l,1,k}$.

In contrast to compute- and memory-reducing methods, DynO focuses on optimizing the device-server communication. To this end, a communication-driven quantization method is proposed. DynO's precision reduction strategy, named ISQuant, entails two key techniques: 1) it applies linear quantization *only* on the intermediate tensors that have to be transferred between the partitions;

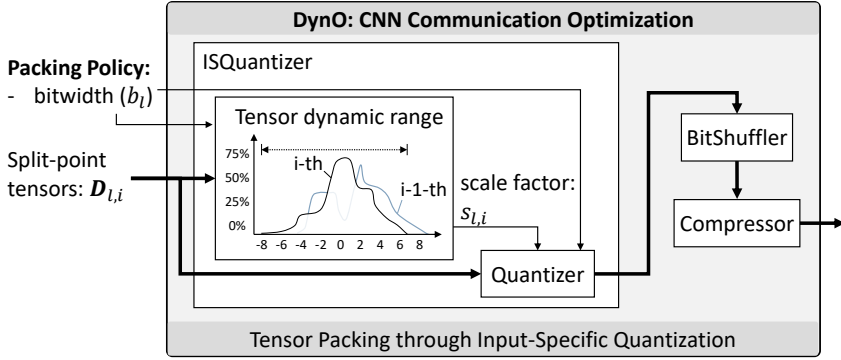


图 3.CNN 通信优化器, CNN-ISPM。

3.3 CNN通信优化

CNN 层操作的张量的大小可能差异很大, 即使对于单输入批次, 许多张量也达到数百 KB。传输这些张量可能很快就会超过加载的好处, 特别是在网络条件较差的情况下。为了克服这个问题并优化两个处理方之间的通信, 我们引入了输入特定的打包模块 (CNN-ISPM)。CNN-ISPM 背后的关键思想是观察到: 1) 对于给定的精度水平, 不同层的输出张量需要不同的精度, 即每个层可以具有不同的位宽, 2) 在任何给定时间点使用的数据表示只需要容纳要传输到远程端的张量值, 即构成所选分割点的依赖项的张量, 以及进一步仅容纳它们针对手头特定输入的值。CNN-ISPM 采用 CNN 专用的三相打包机制 (图 3), 包括: 输入特定精度量化 (ISQuant)、位混洗和无损压缩。

输入特定量化: CNN 中经常使用精确量化来减少内存占用和延迟 [47]。就量化哪些数据而言, 大多数现有方案要么针对权重和激活[15], 要么仅针对所有层的权重[66]。在如何量化方面, 现有的工作通常采用基于块浮点 (也称为动态定点) 的线性量化[14,15,25,33,56], 它规定了模型各层的统一位宽和非统一比例因子。对于第 l 层, 此方法可以表示为 $q_l = \langle b, s_l \rangle$, 其中 b 为位宽, s_l 为确定要保留哪些位的比例因子。

在大多数现有工作中, s_l 的值是在设计时通过估计校准集 (例如目标数据集的子集) 上数据的动态范围来确定的。这种方法受到两个主要因素的限制。首先, 需要很长的分析阶段来确定比例因子, 并强烈假设校准集代表推理时间数据。其次, 在估计的动态范围未捕获当前输入范围的情况下, 这种方法未充分利用所选位宽的代表性范围。图4说明了这种情况。符号 $D_{l,i,k}$ 表示分割点 i 处第 i 个输入的第 k 个依赖张量。估计的动态范围 (灰色虚线) (在设计时用于将比例因子设置为固定值) 不会捕获输入张量 $D_{l,i,k}$ 的实际范围。

与减少计算和内存的方法相比, DynO 专注于优化设备-服务器通信。为此, 提出了一种通信驱动的量化方法。DynO 的精度降低策略名为 ISQuant, 需要两项关键技术: 1) 它仅对必须在分区之间传输的中间张量应用线性量化;

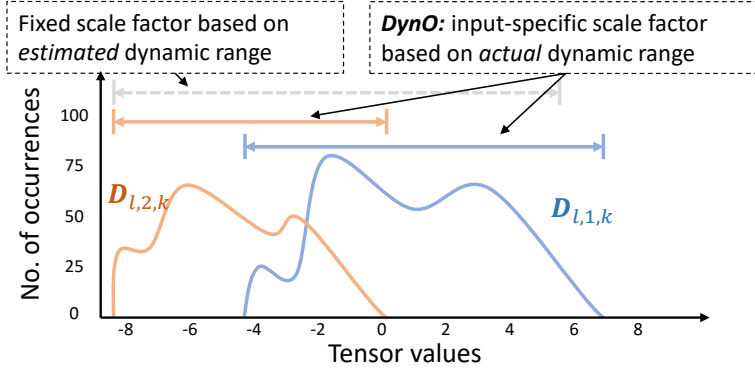


Fig. 4. DynO’s input-specific quantization, ISQuant. We depict three dynamic ranges: 1) the start and end points of the blue and orange curves show the actual min and max values for the two input tensors ($D_{l,1,k}$ and $D_{l,2,k}$, respectively), 2) the estimated dynamic range when using fixed, offline estimation (grey dotted line), and 3) the dynamic range captured by DynO for samples $D_{l,1,k}$ and $D_{l,2,k}$ shown as blue and orange solid lines, respectively.

and 2) it adapts the scale factor in an *input-dependent* (sample-specific) manner. As shown in Fig. 4, this approach enables DynO to tightly follow each tensor’s representational needs through an input-adaptive scale factor (e.g. for both $D_{l,1,k}$ and $D_{l,2,k}$). Formally, we express ISQuant as $q_{l,i,k}^{\text{ISQuant}} = \langle b_l, s_{l,i,k} \rangle$ with a different bitwidth b_l for each layer l and an input-specific (i-th) scale factor $s_{l,i,k}$ for each tensor (k-th) in the split point’s dependencies. In this manner, the scale factor of each tensor is derived at run time to cover the full range of its values (Algorithm 1 lines 2-4). The bitwidth b_l constitutes the *data packing policy* and is dynamically selected by DynO’s scheduler to meet the multi-objective requirements of the target use-case (as detailed in § 3.5).

To demonstrate how ISQuant better captures the dynamic range of the dependency tensors, Fig. 5 shows a comparison of the real and estimated dynamic range for Inception-v3 on ImageNet. For the fixed scale-factor baseline, we divide the validation set into multiple class-balanced calibration sets containing 5% of the samples (20-fold).² For each split point (normalized activation index on the x-axis), the real range boxes represent the distribution of values across all validation set samples. For the estimated range, the boxes indicate the estimates’ distribution across the 20 calibration sets. For conciseness, we show every third split point, with similar findings for the remaining splits.

Based on the figure, we make two key observations. First, there is significant variability in the range estimates across different calibration sets. In this respect, the effectiveness of the baseline relies on the faithfulness of the calibration set with regards to the actual processed data upon deployment. Hence, the selection of the calibration set plays a key role in accurately estimating the dynamic range. Second, we observe a large variation in the tensor values. Across all split points, there is a significant amount of outliers that are not captured by the estimated range (e.g. for the 2nd split point, the 2nd quartile is outside the estimated range and multiple points exceed it by much). As a result, all values that lie above the estimated range (45.35% of the samples in Fig. 5) are clamped and numerical error is induced. In contrast to these, the input-specific approach of ISQuant uses the *real* range of the data to better capture their values when quantizing, thus minimizing the reconstruction error.

²In this experiment, we used 5% of ImageNet’s validation set for calibration and assumed that the rest (i.e. 95%) is the real dataset that would be observed upon deployment. We repeated this process 20 times, each time selecting the 5% calibration set in a random (20-fold cross-validation) manner.

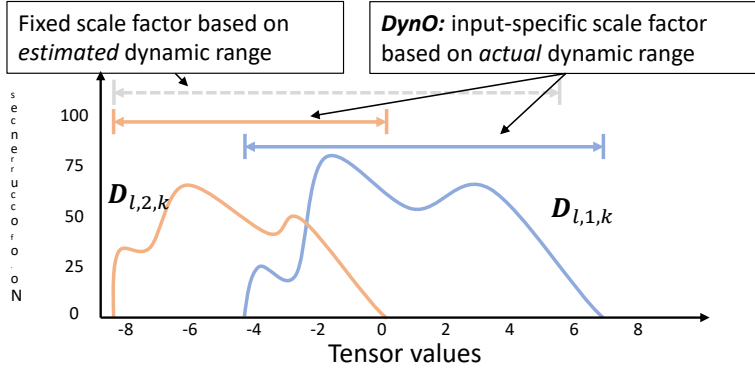


图 4. DynO 的输入特定量化 ISQuant。我们描绘了三个动态范围：1) 蓝色和橙色曲线的起点和终点显示两个输入张量（分别为 $D_{l,1,k}$ 和 $D_{l,2,k}$ ）的实际最小值和最大值，2) 使用固定离线估计时的估计动态范围（灰色虚线），以及 3) DynO 捕获的样本 $D_{l,1,k}$ 和 $D_{l,2,k}$ 的动态范围显示为蓝色和分别为橙色实线。

2) 它以依赖于输入（特定于样本）的方式调整比例因子。如图 4 所示，这种方法使 DynO 能够通过输入自适应比例因子（例如，对于 $D_{l,1,k}$ 和 $D_{l,2,k}$ ）严格遵循每个张量的表示需求。形式上，我们将 ISQuant 表示为 $q_{l,i,k}^{\text{ISQuant}} = \langle b_l, s_{l,i,k} \rangle$ ，每个层 l 具有不同的位宽 b_l ，并且分割点依赖项中的每个张量（第 k 个）具有输入特定的（第 i 个）比例因子 $s_{l,i,k}$ 。通过这种方式，在运行时导出每个张量的比例因子以覆盖其值的整个范围（算法 1 第 2-4 行）。位宽 b_l 构成数据打包策略，并由 DynO 的调度程序动态选择，以满足目标用例的多目标要求（如第 3.5 节中详述）。

为了演示 ISQuant 如何更好地捕获依赖张量的动态范围，图 5 显示了 ImageNet 上 Inception v3 的实际动态范围和估计动态范围的比较。对于固定比例因子基线，我们将验证集划分为包含 5% 样本（20 倍）的多个类平衡校准集。²对于每个分割点（x 轴上的归一化激活指数），真实范围框表示所有验证集样本中值的分布。对于估计范围，方框表示 20 个校准集的估计分布。为了简洁起见，我们显示了每三个分割点，其余分割点也有类似的结果。

根据该图，我们做出两个关键观察。首先，不同校准集的范围估计存在显著差异。在这方面，基线的有效性取决于校准集对部署时实际处理数据的忠实度。因此，校准集的选择对于准确估计动态范围起着关键作用。其次，我们观察到张量值存在很大变化。在所有分割点中，存在大量未在估计范围内捕获的异常值（例如，对于第二个分割点，第二个四分位数位于估计范围之外，并且多个点超出了估计范围很多）。结果，所有高于估计范围的值（图 5 中样本的 45.35%）都被钳位，并产生数值误差。与这些相反，ISQuant 的输入特定方法使用数据的真实范围在量化时更好地捕获它们的值，从而最大限度地减少重建误差。

。

²In this experiment, we used 5% of ImageNet's validation set for calibration and assumed that the rest (*i.e.* 95%) is the real dataset that would be observed upon deployment. We repeated this process 20 times, each time selecting the 5% calibration set in a random (20-fold cross-validation) manner.

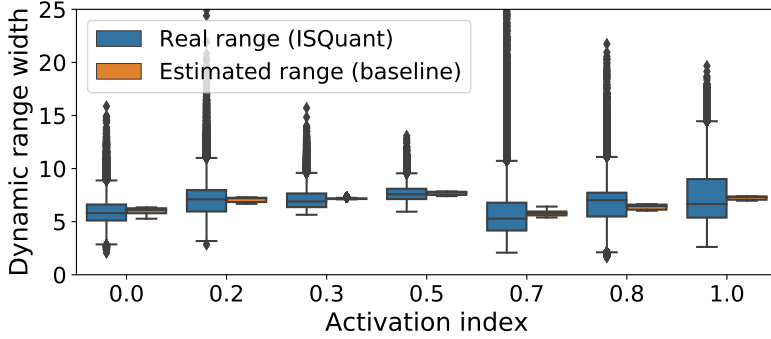


Fig. 5. Distribution of estimated range (5% calibration set) vs the real observed range for Inception-v3 on ImageNet.

Overall, our quantization scheme maximizes the utilization of the selected bitwidth in a per-input sample basis and isolates the approximation to the split-point dependency tensors. Both of these characteristics enable DynO to push the data representation to lower precisions and significantly minimize the data transfer cost. Moreover, while most existing works [12, 15, 16] employ a post-quantization retraining step in order to minimize any induced accuracy losses, our approach – that maintains the rest of the CNN in full precision – enables us to skip the costly retraining step and requires no availability of the training data. As shown in § 4.2.2, ISQuant can lead down to 4 and 5 bits of precision, in most cases, with very low impact on the overall model accuracy.

Bit shuffling: This step transposes the transferred data matrix, so that all least significant bits lie in the same row [44]. This rearrangement allows the elimination of the compute-heavy Huffman coding (as in GZip) in favor of a faster LZ77-class compressor, such as LZ4.

Compression: To further exploit the redundancy of the transferred data, CNN-ISPM introduces a lossless compression stage. Internally, this stage employs LZ4, a fast lossless compression algorithm. The precedence of ISQuant and bit shuffling results in a significant reduction in the data entropy. In this manner, the resulting sizes are up to 60× smaller than the original tensors (§ 4.2.1). In § 4.2.1, we present a comparative evaluation of different compressors, justifying the selection of LZ4.

Implementation. Prior to transmitting the tensor dependencies, the client follows the pipeline depicted in Fig. 3 and Algorithm 1. For CNN-ISPM not to starve the client’s resources, we apply the following optimizations. ISQuant is mapped to the client’s mobile GPU (if available), with the min/max operations (lines 2-3) vectorized, and the quantization of each element of tensor $D_{l,i,k}$ performed in parallel (lines 5-7). The bit shuffling and compression phases (lines 9-10) are mapped to the CPU utilizing the SIMD instructions of the target processor (e.g. NEON on ARM cores). Overall, CNN-ISPM is run as a separate thread enabling the pipelining of inference and packing. On the server side, the received data are first decompressed and then dequantized back to the original bitwidth, before being injected to the model to resume the inference computation. We note that this approach has very little impact on the memory consumption compared to local execution, as DynO compresses only the activations that need to be transferred. As such, the memory requirements for the model and the intermediate results are not affected.

With CNN-ISPM, the amount of packing is configurable. If the network conditions are sufficiently good to support the application’s deadlines, higher bitwidths can be used to maximize accuracy. As the network conditions degrade, DynO’s scheduler may choose a more aggressive packing policy by quantizing down to smaller bitwidths. The bitwidth selection process will be described in § 3.5.

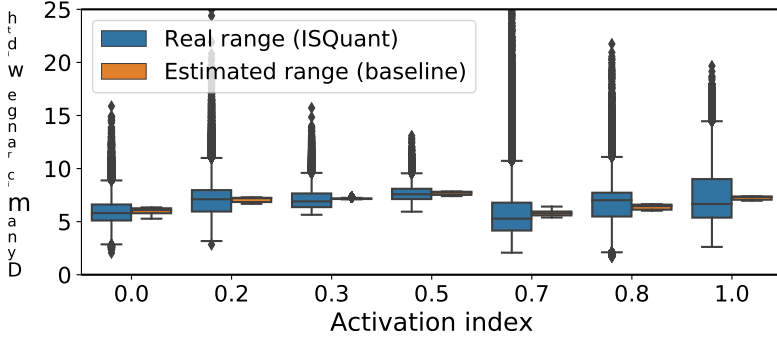


图 5. ImageNet 上 Inception-v3 的估计范围（5% 校准集）与实际观测范围的分布

总体而言，我们的量化方案在每个输入样本的基础上最大化了所选位宽的利用率，并隔离了分裂点相关张量的近似值。这两个特性使 DynO 能够将数据表示降低到更低的精度，并显著降低数据传输成本。此外，虽然大多数现有工作 [12,15,16] 采用后量化再训练步骤，以尽量减少任何引起的精度损失，但我们的方法（保持 CNN 其余部分的精度）使我们能够跳过成本高昂的再训练步骤，并且不需要训练数据的可用性。如第 4.2.2 节所示，在大多数情况下，ISQuant 可以将精度降低至 4 位和 5 位，对整体模型精度的影响非常小。

位改组：此步骤转置传输的数据矩阵，以便所有最低有效位位于同一行[44]。这种重新安排可以消除计算量大的霍夫曼编码（如 GZip），有利于更快的 LZ77 级压缩器，例如 LZ4。

压缩：为了进一步利用传输数据的冗余，CNN-ISP M 引入了无损压缩阶段。在内部，该阶段采用 LZ4，一种快速无损压缩算法。ISQuant 和位混洗的优先级导致数据熵显著降低。通过这种方式，生成的大小最多比原始张量小 60×（第 4.2.1 节）。在第 4.2.1 节中，我们对不同压缩机进行了比较评估，证明了选择 LZ4 的合理性。

执行。在传输张量依赖关系之前，客户端遵循图 3 和算法 1 中描述的流程。为了 CNN-ISP M 不耗尽客户端的资源，我们应用了以下优化。ISQuant 映射到客户端的移动 GPU（如果可用），最小/最大运算（第 2-3 行）矢量化，并且张量 $D_{l,i,k}$ 每个元素的量化并行执行（第 5-7 行）。位改组和压缩阶段（第 9-10 行）利用目标处理器（例如 ARM 内核上的 NEON）的 SIMD 指令映射到 CPU。总体而言，CNN-ISP M 作为单独的线程运行，支持推理和打包的流水线操作。在服务器端，接收到的数据首先被解压缩，然后反量化回原始位宽，然后注入模型以恢复推理计算。我们注意到，与本地执行相比，这种方法对内存消耗的影响非常小，因为 DynO 仅压缩需要传输的激活。因此，模型和中间结果的内存需求不会受到影响。

使用 CNN-ISP M，打包量是可配置的。如果网络条件足以支持应用程序的最后期限，则可以使用更高的位宽来最大限度地提高准确性。随着网络条件的恶化，DynO 的调度程序可能会通过量化到更小的位宽来选择更积极的打包策略。位宽选择过程将在第 3.5 节中描述。

Algorithm 1: CNN-ISP’s communication optimization

Input: Selected split point l and bitwidth b_l (*packing policy*)

k -th dependency tensor $D_{l,i,k}$ for the i -th input at split point l

Output: k -th packed tensor to be transferred $D_{l,i,k}^{\text{packed}}$

```
1 /* - - - Phase 1 - Input-Specific Quantization - - - */
2  $val_{\min} \leftarrow \min(D_{l,i,k})$ 
3  $val_{\max} \leftarrow \max(D_{l,i,k})$ 
4  $s_{l,i,k} \leftarrow \log_2 \left( \frac{2^{b_l} - 1}{val_{\max} - val_{\min}} \right)$ 
5 for  $d$  in  $D_{l,i,k}$  do // loop over the data of the  $k$ -th dependency tensor
6    $d^{\text{quant}} \leftarrow (d - val_{\min}) \cdot 2^{s_{l,i,k}}$ 
7 end
8  $D_{l,i,k}^{\text{quant}} \leftarrow \lfloor \forall d^{\text{quant}} \rfloor$ 
9  $D_{l,i,k}^{\text{bitshuffled}} \leftarrow \text{BitShuffle}(D_{l,i,k}^{\text{quant}})$  /* - - Phase 2 - Bit Shuffling - - */
10  $D_{l,i,k}^{\text{packed}} \leftarrow \text{Compress}(D_{l,i,k}^{\text{bitshuffled}})$  /* - Phase 3 - Compression - */
```

3.4 Dynamic Profiler

After identifying the dependencies for each possible split point s , DynO needs to make decisions about where to split the CNN and how much packing c to apply on the transferred dependencies. These decisions greatly affect the inference i) latency, ii) throughput, iii) accuracy and iv) cost of operation at both ends. DynO makes these decisions by estimating the device, network and server times, as well as the expected accuracy, for each possible configuration $\langle s, c \rangle$. The *Profiler* (Fig. 1) tracks these key performance metrics at two separate phases: i) *Offline* and ii) *Run-time*.

Offline profiling: Before deploying a CNN to a set of devices, an initial calibration round is performed to initialize the profiling metrics. First, we note that the size of the data dependencies $d_{\langle s,c \rangle}$ and the accuracy loss $Acc_{\langle s,c \rangle}$ are *not device-dependent*. Therefore, these can be profiled *once for a given model*. In our implementation, the profiler calculates these values using the target task’s validation set. Upon deployment, the profiler estimates the CNN computation times that *are* device-dependent. This is accomplished by going through a calibration set³ once, for each available processing unit (e.g. CPU, GPU, NPU) measuring the average time to execute each layer (input-independent) and to compress their dependencies (input-dependent). These are treated as initial values, to be later updated at run time.

Run-time profiling: At run time, the profiler keeps updating the estimated latencies by taking into account the device load and the networking conditions. To estimate the real-world computation latency, the profiler logs the processing unit and memory utilization just before an inference is performed. During inference, DynO records the compute times up to split point s and calculates a *scaling factor* $SF = \frac{T_{\text{real}}(s)}{T_{\text{offline}}(s)}$ between the measured time and the offline estimate. Next, the profiler interprets the scaling factor as a proxy of the device load and uses it to estimate the runtime all other possible splits, e.g. $SF \cdot T_{\langle s' \rangle}^{\text{offline}}$ for new split s' .

To estimate the *network transfer time*, DynO’s profiler monitors the device’s network bandwidth (B) and latency (L). The transfer time is $L + \frac{d_{\langle s,c \rangle}}{B}$ where $d_{\langle s,c \rangle}$ is the data size to be transferred given split s and packing c . As networking fluctuates, two moving averages are maintained: a real-time estimate and a historical moving average, $\langle L^{\{\text{rt},h\}}, B^{\{\text{rt},h\}} \rangle$. Real-time estimates are obtained only if there are transfers in the last 5 minutes. If no such information exists, the historical averages for the same network type are used.

³Sampled uniformly across labels from the target task’s validation set.

Algorithm 1: CNN-ISP’s communication optimization

Input: Selected split point l and bitwidth b_l (*packing policy*)

k -th dependency tensor $D_{l,i,k}$ for the i -th input at split point l

Output: k -th packed tensor to be transferred $D_{l,i,k}^{\text{packed}}$

```
1 /* --- Phase 1 - Input-Specific Quantization --- */
2  $val_{\min} \leftarrow \min(D_{l,i,k})$ 
3  $val_{\max} \leftarrow \max(D_{l,i,k})$ 
4  $s_{l,i,k} \leftarrow \log_2 \left( \frac{2^{b_l}-1}{val_{\max}-val_{\min}} \right)$ 
5 for  $d$  in  $D_{l,i,k}$  do // loop over the data of the  $k$ -th dependency tensor
6    $d^{\text{quant}} \leftarrow (d - val_{\min}) \cdot 2^{s_{l,i,k}}$ 
7 end
8  $D_{l,i,k}^{\text{quant}} \leftarrow \lfloor d^{\text{quant}} \rfloor$ 
9  $D_{l,i,k}^{\text{bitshuffled}} \leftarrow \text{BitShuffle}(D_{l,i,k}^{\text{quant}})$  /* - Phase 2 - Bit Shuffling - */
10  $D_{l,i,k}^{\text{packed}} \leftarrow \text{Compress}(D_{l,i,k}^{\text{bitshuffled}})$  /* - Phase 3 - Compression - */
```

3.4 动态分析器

在确定每个可能的分割点 s 的依赖关系后，DynO 需要决定在何处分割 CNN 以及对传输的依赖关系应用多少打包 c 。这些决策极大地影响了推理 i) 延迟、ii) 吞吐量、iii) 准确性和 iv) 两端的操作成本。DynO 通过估计每种可能的配置 $\langle s, c \rangle$ 的设备、网络和服务器时间以及预期精度来做出这些决策。Profiler（图 1）在两个不同的阶段跟踪这些关键性能指标：i) 离线阶段和 ii) 运行时阶段。

离线分析：在将 CNN 部署到一组设备之前，会执行一轮初始校准来初始化分析指标。首先，我们注意到数据依赖项 $d_{\langle s, c \rangle}$ 的大小和准确性损失 $Acc_{\langle s, c \rangle}$ 不依赖于设备。因此，可以针对给定模型对这些进行分析一次。在我们的实现中，探查器使用目标任务的验证集来计算这些值。部署后，分析器会估计与设备相关的 CNN 计算时间。这是通过对每个可用处理单元（例如 CPU、GPU、NPU）执行一次校准集³来测量执行每个层（与输入无关）和压缩其依赖性（与输入相关）的平均时间来实现的。这些被视为初始值，稍后在运行时更新。

运行时分析：在运行时，分析器会考虑设备负载和网络条件，不断更新估计的延迟。为了估计现实世界的计算延迟，分析器会在执行推理之前记录处理单元和内存利用率。在推理过程中，DynO 记录直到分割点 s 的计算时间，并计算测量时间和离线估计之间的缩放因子 $SF = \frac{T_{\text{real}}(s)}{T_{\text{offline}}(s)}$ 。接下来，分析器将缩放因子解释为设备负载的代理，并使用它来估计所有其他可能的拆分的运行时间，例如 $SF \cdot T_{(s')}^{\text{offline}}$ 用于新拆分 s' 。

为了估计网络传输时间，DynO 的分析器会监控设备的网络带宽 (B) 和延迟 (L)。传输时间为 $L + \frac{d_{\langle s, c \rangle}}{B}$ ，其中 $d_{\langle s, c \rangle}$ 是给定分割 s 和打包 c 时要传输的数据大小。当网络波动时，会维护两个移动平均线：实时估计和历史移动平均线， $\langle L^{\{\text{rt}, h\}}, B^{\{\text{rt}, h\}} \rangle$ 。仅当最后 5 分钟内有转账时才能获得实时估算。如果不存在此类信息，则使用同一网络类型的历史平均值。

³Sampled uniformly across labels from the target task’s validation set.

Algorithm 2: Operation of dynamic scheduler upon invocation

Input: Space of candidate configurations Σ

Prioritised hard constraints $\langle C_1, C_2, \dots, C_n \rangle$

Prioritised soft targets $\langle O_1, O_2, \dots, O_{|\mathcal{M}|} \rangle$

Current network conditions $net = \langle L, B \rangle$

Current device and server loads $l^{\{\text{dev}, \text{server}\}}$

Profiler data prf

Output: Selected configuration $\langle s^*, c^* \rangle$

```
1  $prf \leftarrow \text{UpdateTimings}(prf, net, l^{\text{dev}}, l^{\text{server}})$ 
2  $\Sigma^{\text{feasible}} \leftarrow \Sigma$ 
3 foreach  $C_i \in \langle C_1, C_2, \dots, C_n \rangle$  do // discard infeasible configuration
4    $\Sigma^{\text{feasible}} \leftarrow \text{DiscardInfeasibleConfigs}(prf, C_i, \Sigma^{\text{feasible}})$ 
5    $\hookrightarrow \text{VecCheckCond}(prf, \Sigma^{\text{feasible}}(:, M_i), op_i, thr_i) \quad \forall i \in [1, n]$ 
6 end
7  $\langle s^*, c^* \rangle \leftarrow \text{OptUserSoftTargets}(prf, \langle O_1, O_2, \dots, O_{|\mathcal{M}|} \rangle, \Sigma^{\text{feasible}})$ 
8  $\hookrightarrow \text{VecMax/Min}(prf, \Sigma^{\text{feasible}}(:, M_i), op_i) \quad \forall i \in [1, |\mathcal{M}|]$ 
```

3.5 Dynamic Scheduler

Given the output of the profiler, the *dynamic scheduler* is responsible for deciding how to distribute the CNN computation and tune the data packing so as to satisfy the application requirements. The dynamic aspect is particularly important for mobile devices where connectivity and load conditions can frequently change (e.g. move from WiFi to 3G). To capture diverse tasks, the scheduler allows developers to define a combination of *hard constraints* (e.g. inference latency ≤ 100 ms) and *soft optimization targets* (e.g. minimize cloud costs) on a set of metrics. In our current implementation, the set of metrics includes $\mathcal{M} = \langle \text{latency}, \text{throughput}, \text{server cost}, \text{device cost}, \text{accuracy} \rangle$. In DynO, we interpret server (cloud) and device cost as the execution time on the respective side. This formulation can cover a wide range of use-cases. For example, in latency-bound tasks [32, 35, 61], one might want to onload as much as possible to the client as long as the inference latency is below a threshold thr_{lat} .

Formally, we capture a hard constraint as $C = \langle m, op, thr \rangle$ where $m \in \mathcal{M}$ is a metric, op is an operator, e.g. $\leq$, and thr is a given threshold value. Similarly, we define a soft optimization target as $O = \langle m, min || max || val \rangle$ where a given metric is either maximized, minimized or close to a given value. To capture the importance of each metric, we adopt a multi-objective formulation, where the user supplies a list of *prioritized* hard constraints $\langle C_1, \dots, C_n \rangle$ and a list of ordered soft optimization targets $\langle O_1, \dots, O_{|\mathcal{M}|} \rangle$.

Algorithm 2 describes the operation of the scheduler. First, the scheduler uses the estimated network conditions, and device and server loads to update the profiler parameters (line 1). Given the list of *prioritized* hard constraints, the scheduler iteratively eliminates all (split, packing) configurations $\langle s, c \rangle$ that violate the constraints in that order (lines 3-6). If at any point no configuration satisfies the constraints, the closest configuration is immediately returned (i.e. fall back to best-effort). If more than one configuration satisfy the constraints, the soft targets are used to sort them (line 7). Finally, the best configuration is returned by the scheduler, with split s^* and packing policy c^* used to configure the partitioner and CNN-ISPM.

Implementation. The scheduler is deployed on the client where the data reside and the inference is initiated. To minimize resource usage, we vectorize the comparison and max/min operations (lines 4-5, 7-8) to utilize SIMD instructions. This way, the scheduler executes in max 14 ms (11 ms geo. mean across examined CNNs on ImageNet) on Jetson's CPU with a few KB of memory usage (setup details in § 4.1). Finally, the scheduler's overhead is amortized over multiple inferences as the scheduler is only re-invoked when the profiler metrics change by more than 5%.

Algorithm 2: Operation of dynamic scheduler upon invocation

Input: Space of candidate configurations Σ

Prioritised hard constraints $\langle C_1, C_2, \dots, C_n \rangle$

Prioritised soft targets $\langle O_1, O_2, \dots, O_{|M|} \rangle$

Current network conditions $net = \langle L, B \rangle$

Current device and server loads $l^{\{dev, server\}}$

Profiler data prf

Output: Selected configuration $\langle s^*, c^* \rangle$

```
1  $prf \leftarrow \text{UpdateTimings}(prf, net, l^{dev}, l^{server})$ 
2  $\Sigma^{feasible} \leftarrow \Sigma$ 
3 foreach  $C_i \in \langle C_1, C_2, \dots, C_n \rangle$  do // discard infeasible configuration
4    $\Sigma^{feasible} \leftarrow \text{DiscardInfeasibleConfigs}(prf, C_i, \Sigma^{feasible})$ 
5    $\nabla \text{VecCheckCond}(prf, \Sigma^{feasible}(:, M_i), op_i, thr_i) \quad \forall i \in [1, n]$ 
6 end
7  $\langle s^*, c^* \rangle \leftarrow \text{OptUserSoftTargets}(prf, \langle O_1, O_2, \dots, O_{|M|} \rangle, \Sigma^{feasible})$ 
8  $\nabla \text{VecMax/Min}(prf, \Sigma^{feasible}(:, M_i), op_i) \quad \forall i \in [1, |M|]$ 
```

3.5 动态调度器

给定分析器的输出，动态调度器负责决定如何分配 CNN 计算并调整数据打包以满足应用程序需求。动态方面对于连接和负载条件可能频繁变化（例如从 WiFi 移动到 3G）的移动设备尤其重要。为了捕获不同的任务，调度程序允许开发人员在一组指标上定义硬约束（例如推理延迟 ≤ 100 毫秒）和软优化目标（例如最小化云成本）的组合。在我们当前的实现中，这组指标包括 $M = \langle \text{延迟、吞吐量、服务器成本、设备成本、准确性} \rangle$ 。在 DynO 中，我们将服务器（云）和设备成本解释为各自一侧的执行时间。该公式可以涵盖广泛的用例。例如，在延迟受限的任务中 [32,35,61]，只要推理延迟低于阈值 thr_{lat} ，人们可能希望向客户端加载尽可能多的数据。

形式上，我们将硬约束捕获为 $C = \langle m, op, thr \rangle$ ，其中 $m \in M$ 是一个度量， op 是一个运算符，例如 $\leq$ 和 thr 是给定的阈值。类似地，我们将软优化目标定义为 $O = \langle m, min || max || val \rangle$ ，其中给定指标要么最大化、最小化或接近给定值。为了捕获每个指标的重要性，我们采用多目标公式，其中用户提供优先级硬约束列表 $\langle C_1, \dots, C_n \rangle$ 和有序软优化目标列表 $\langle O_1, \dots, O_{|M|} \rangle$ 。

算法 2 描述了调度程序的操作。首先，调度程序使用估计的网络条件以及设备和服务器负载来更新分析器参数（第 1 行）。给定优先级硬约束列表，调度程序迭代地消除所有违反该顺序约束的（分割、打包）配置 $\langle s, c \rangle$ （第 3-6 行）。如果在任何时候没有配置满足约束，则立即返回最接近的配置（即回退到尽力而为）。如果多个配置满足约束，则使用软目标对它们进行排序（第 7 行）。最后，调度器返回最佳配置，其中分割 s^* 和打包策略 c^* 用于配置分区器和 CNN-ISP。

执行。调度器部署在数据所在的客户端并启动推理。为了最大限度地减少资源使用，我们对比较和最大/最小操作（第 4-5、7-8 行）进行矢量化以利用 SIMD 指令。这样，调度程序在 Jetson 的 CPU 上的执行时间最多为 14 毫秒（在 ImageNet 上检查的 CNN 中平均为 11 毫秒），内存使用量为几 KB（设置详细信息参见第 4.1 节）。最后，调度程序的开销通过多次推理进行摊销，因为只有当分析器指标变化超过 5% 时才会重新调用调度程序。

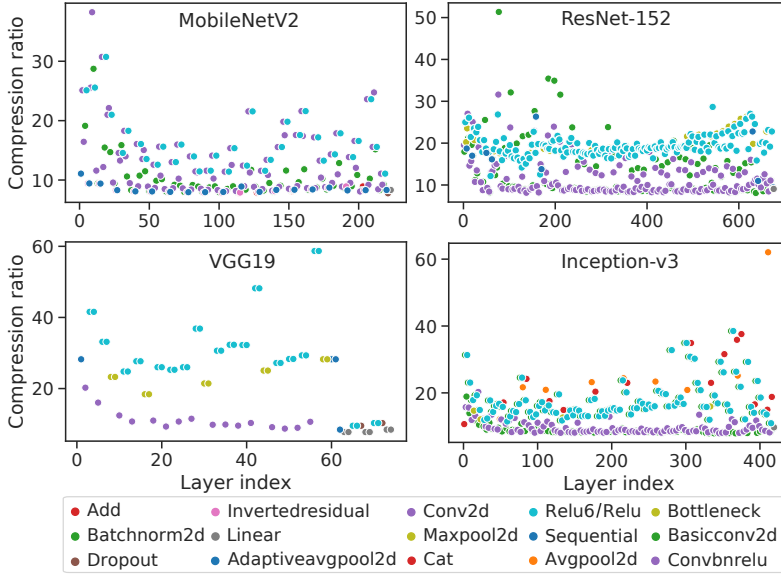


Fig. 6. Compression ratio for each model’s layers with CNN-ISPM (using 4 bits). Colors represent different layer types.

Table 2. Target Platforms

Platform	Processor	Memory	GPU
Server-Desktop	Intel i7-7820X	128GB DDR4	Nvidia GTX1080Ti
Client-Jetson Xavier	8-core ARM-Karmel v8.2	16GB LPDDR4x	512-core Volta

Pipelining. To further optimize DynO’s throughput under normal server load without backpressure, we apply pipelining. In this respect, the compression, network and inference stages run as separate parallel threads. When the scheduler is instructed to maximize inference throughput, it aims to minimize the maximum latency across these stages, to balance the client, server and network times.

4 EVALUATION

4.1 Experimental Setup

In our experiments, we use a high-end desktop as the server and Jetson Xavier AGX as the client, both connected over Gigabit Ethernet (details are provided at Table 2). We specifically chose Jetson as our device due to its compact form factor, versatility and adjustable power consumption and computational performance, which makes it ideal for diverse use-cases. To emulate devices of different capabilities, we adjust the TDP and clock rate of the CPU and GPU cores and test against three different power profiles: 1) *30W*, using AGX’s MAXN mode (e.g. for drones and robots), 2) *10W* (e.g. for smart cameras), 3) *underclocked 10W* (e.g. smartphones and tablets). To better control network conditions, we simulate them using the average bandwidth and latency across national carriers [48] for 3G and 4G networks. For local-area connections (Gb Ethernet 802.3, WiFi-5 802.11ac), we use the nominal speeds of the respective protocol.

Choice of neural networks: We developed DynO on top of *PyTorch* (1.1.0) and experimented with four ImageNet-pretrained models from *torchvision* (0.3.0): Inception-v3 [57], ResNet-152 [20], VGG19 [55] and MobileNetV2 [54]. Across all experiments, we use a batch size of 1. With respect to

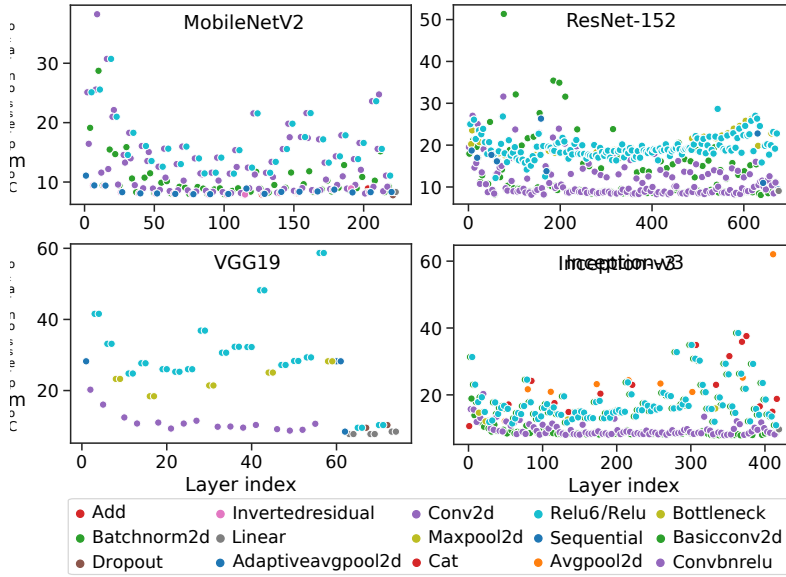


图 6. CNN-ISPM 的每个模型层的压缩率（使用 4 位）。颜色代表不同的图层类型。

表 2. 目标平台

Platform	Processor	Memory	GPU
Server-Desktop	Intel i7-7820X	128GB DDR4	Nvidia GTX1080Ti
Client-Jetson Xavier	8-core ARM-Karmel v8.2	16GB LPDDR4x	512-core Volta

管道化。为了进一步优化 DynO 在正常服务器负载下无背压的吞吐量，我们应用了流水线。在这方面，压缩、网络和推理阶段作为单独的并行线程运行。当调度程序被指示最大化推理吞吐量时，它的目标是最小化这些阶段的最大延迟，以平衡客户端、服务器和网络时间。

4 评价

4.1 实验设置

在我们的实验中，我们使用高端桌面作为服务器，Jetson Xavier AGX 作为客户端，两者都通过千兆以太网连接（详细信息参见表 2）。我们特别选择 Jetson 作为我们的设备，因为它具有紧凑的外形、多功能性以及可调节的功耗和计算性能，这使其成为各种用例的理想选择。为了模拟不同功能的设备，我们调整 CPU 和 GPU 内核的 TDP 和时钟速率，并针对三种不同的功率配置进行测试：1) 30W，使用 AGX 的 MAXN 模式（例如用于无人机和机器人），2) 10W（例如用于智能相机），3) 降频 10W（例如智能手机和平板电脑）。为了更好地控制网络条件，我们使用 3G 和 4G 网络的全国运营商 [48] 的平均带宽和延迟来模拟它们。对于本地连接（Gb 以太网 802.3、WiFi-5 802.11ac），我们使用相应协议的标称速度。

神经网络的选择：我们在 PyTorch (1.1.0) 之上开发了 DynO，并尝试了 torchvision (0.3.0) 的四种 ImageNet 预训练模型：Inception-v3 [57]、ResNet-152 [20]、VGG19 [55] 和 MobileNetV2 [54]。在所有实验中，我们使用批量大小 1。

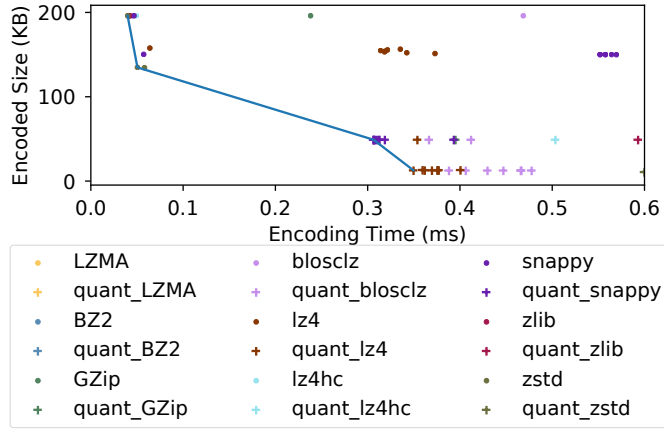


Fig. 7. Time vs size of encoding 200KB ReLU activations, without and with 4-bit ISQuant (denoted with the ‘quant’ prefix). Blue line indicates the pareto front of the best size/time tuples.

input size, Inception-v3 receives $3 \times 299 \times 299$, while the other three models receive $3 \times 224 \times 224$. These models represent a diverse range of CNNs with distinct architectures and sizes, from MobileNetV2 with 0.33 GFLOPs and 3.5M parameters, to VGG19 with 20.2 GFLOPs and 143M parameters. Despite the significant progress in model compression and efficient CNN design, existing efforts have focused on a few popular datasets and tasks. In real-world scenarios, where proprietary datasets and complex tasks are targeted, deriving an efficient model *without* penalizing the accuracy is often not possible. Hence, system designers tend to end up with resource-efficient, but less accurate CNNs. As such, with the exception of MobileNetV2, we intentionally focus on larger networks, where server-assisted execution is more valuable, since these generally offer higher and previously unattainable accuracies on embedded devices. Moreover, ML-powered app developers are reported to deploy DNN models which can be found readily pretrained, rather than architectures at the bleeding edge of deep learning research [2]. As such, we include the widely available VGG network in our experiment, despite the fact that it is not anymore in the latency-accuracy Pareto frontier.

4.2 Compression Analysis

We study two main aspects of the packing module, CNN-ISPM: i) *which layers* are good candidates for compression and *how much* and *how fast* they can be compressed; and ii) *what is the precision limit* we can reach with minimal degradation of the task’s accuracy.

4.2.1 Compression ratio. We started our experiments with two assumptions: i) the widely used ReLU [46] activations zero out negative values and thus tend to have high sparsity [52]; ii) highly sparse data tend to be highly compressible. To select the best compression scheme, we evaluated the compression ratio and latency of multiple compression schemes when compressing ReLU activations, with and without 4-bit ISQuant. Fig. 7 presents the obtained results when compressing a mid-sized ReLU layer of ResNet-18 over 100 inferences. Lossless compression alone can already provide a 30% reduction with less than 0.1 ms encoding time. Nonetheless, with additional 4-bit quantization we witness an increase in the compression ratio, with up to 95% reduction in size. These results indicate that the ISQuant and LZ4 combination is the best performing, reducing 200KB activations by up to 95% in under 0.32 ms and thus use it for the remaining experiments.

To validate our assumption that ReLUs are indeed within the most compressible components of CNNs, we collected the output tensors of every layer in our evaluated networks and applied

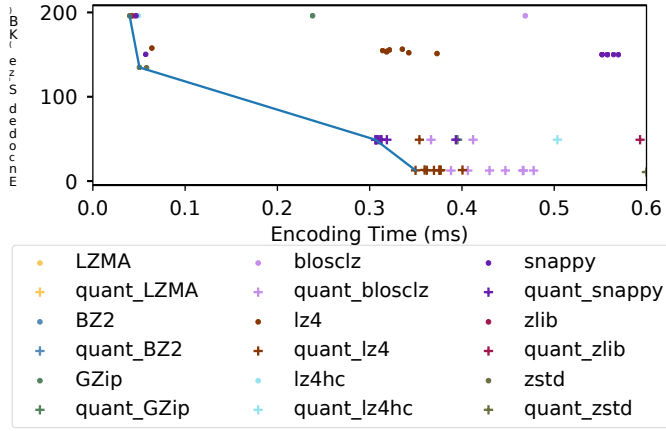


图 7. 编码 200KB ReLU 激活的时间与大小，不带和带 4 位 ISQuant（用“quant”前缀表示）。蓝线表示最佳大小/时间元组的帕累托前沿。

输入大小，Inception-v3 接收 $3 \times 299 \times 299$ ，而其他三个模型接收 $3 \times 224 \times 224$ 。这些模型代表了具有不同架构和大小的各种 CNN，从具有 0.33 GFLOP 和 3.5M 参数的 MobileNetV2，到具有 20.2 GFLOP 和 143M 参数的 VGG19。尽管在模型压缩和高效 CNN 设计方面取得了重大进展，但现有的工作仍集中在一些流行的数据集和任务上。在现实场景中，以专有数据集和复杂任务为目标，在不影响准确性的情况下导出有效的模型通常是不可能的。因此，系统设计者往往会得到资源高效但精度较低的 CNN。因此，除了 MobileNetV2 之外，我们有意关注更大的网络，其中服务器辅助执行更有价值，因为这些网络通常在嵌入式设备上提供更高且以前无法达到的精度。此外，据报道，基于 ML 的应用程序开发人员部署了易于预训练的 DNN 模型，而不是深度学习研究前沿的架构 [2]。因此，我们在实验中纳入了广泛使用的 VGG 网络，尽管事实上它不再处于延迟精度帕累托前沿。

4.2 压缩分析

我们研究了打包模块 CNN-ISPM 的两个主要方面：i) 哪些层适合压缩，以及它们可以压缩的程度和速度；ii) 在任务准确度下降最小的情况下我们可以达到的精确度极限是多少。

4.2.1 压缩比。我们从两个假设开始我们的实验：i) 广泛使用的 ReLU [46] 激活将负值归零，因此往往具有高稀疏性 [52]；ii) 高度稀疏的数据往往具有高度可压缩性。为了选择最佳的压缩方案，我们评估了压缩 ReLU 激活时多种压缩方案的压缩率和延迟，无论是否使用 4 位 ISQuant。图 7 显示了压缩 ResNet-18 的中等大小 ReLU 层超过 100 个推理时获得的结果。仅无损压缩就可以在编码时间不到 0.1 毫秒的情况下实现 30% 的压缩。尽管如此，通过额外的 4 位量化，我们看到压缩比有所提高，大小减少了 95%。这些结果表明 ISQuant 和 LZ4 组合性能最佳，在 0.32 毫秒内将 200KB 激活减少高达 95%，因此将其用于其余实验。

为了验证我们的假设，即 ReLU 确实位于 CNN 中最可压缩的组件内，我们收集了评估网络中每一层的输出张量并应用

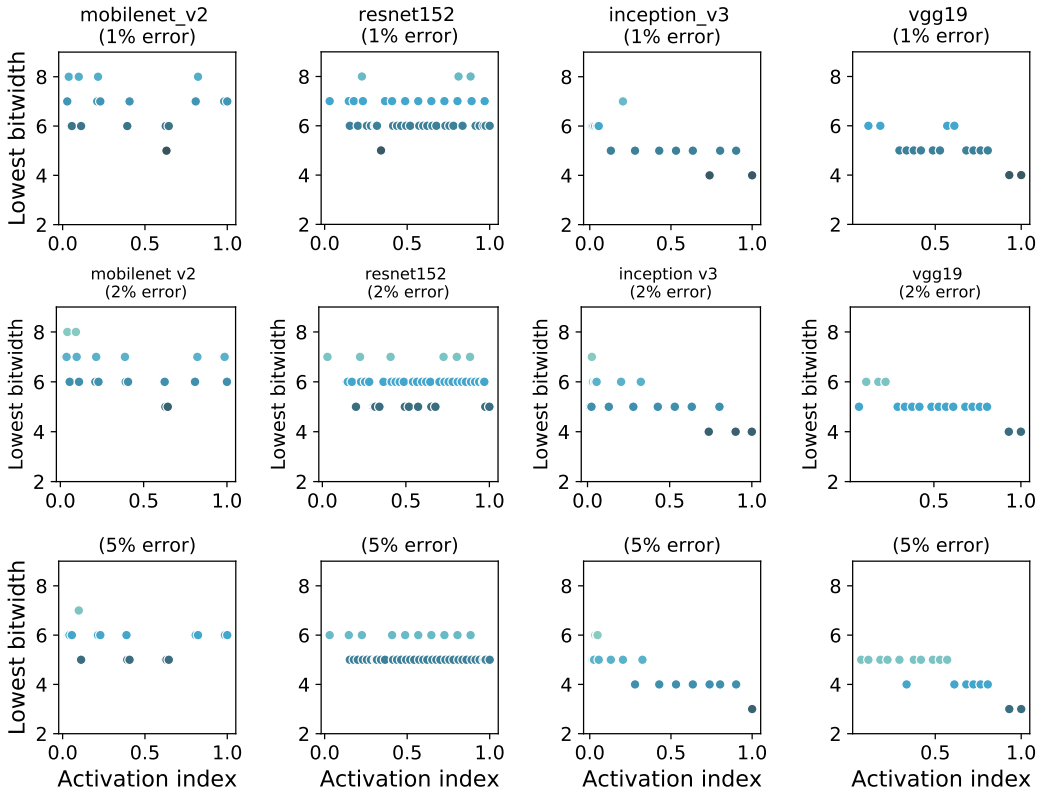


Fig. 8. Best ISQuant bitwidth per layer with a max of 1, 2 and 5 pp accuracy drop. Layer indexes are normalized by the number of layers of each CNN. Most CNNs can be quantized to 3-8 bits with minimal accuracy drop.

CNN-ISPM (Fig. 6). ReLU outputs, spread across the depth of the CNN, consistently yield among the highest compression ratios, reaching up to 60× for VGG19. The trend is similar for the MobileNetV2 and Inception blocks, ending in one or multiple ReLUs.

Takeaways: *Activations such as ReLU are widely spread across popular CNNs and their outputs are prominent candidates for compression, delivering significant size reduction with minimal overhead when using input-specific quantization, bit shuffling and LZ4.*

4.2.2 Accuracy sensitivity to compression. In addition to the potential space savings of packing, DynO requires estimates of its impact on accuracy to better partition and schedule execution. To achieve this, we apply CNN-ISPM to the dependencies of each possible split and measure the impact on ImageNet’s validation accuracy. Since DynO technically allows partitioning the graph at any layer, this process can become time consuming for larger networks. Driven by our previous results, we reduce the partition search space to layers that only have ReLU dependencies. On average, this heuristic reduces the search space by 81% across the examined CNNs, greatly diminishing both the offline profiling and scheduler runtime. Our packing algorithm is still applicable to other activations, but yields different compression dynamics. The Swish function, for example, would show gains from the smaller range of values towards $-\infty$ instead of ReLU’s hard zero bound.

Fig. 8 shows the lowest bitwidth achievable by ISQuant for an accuracy degradation of 1, 2 and 5 percentage points (pp). Rows represent levels of degradation allowance and columns different CNNs.

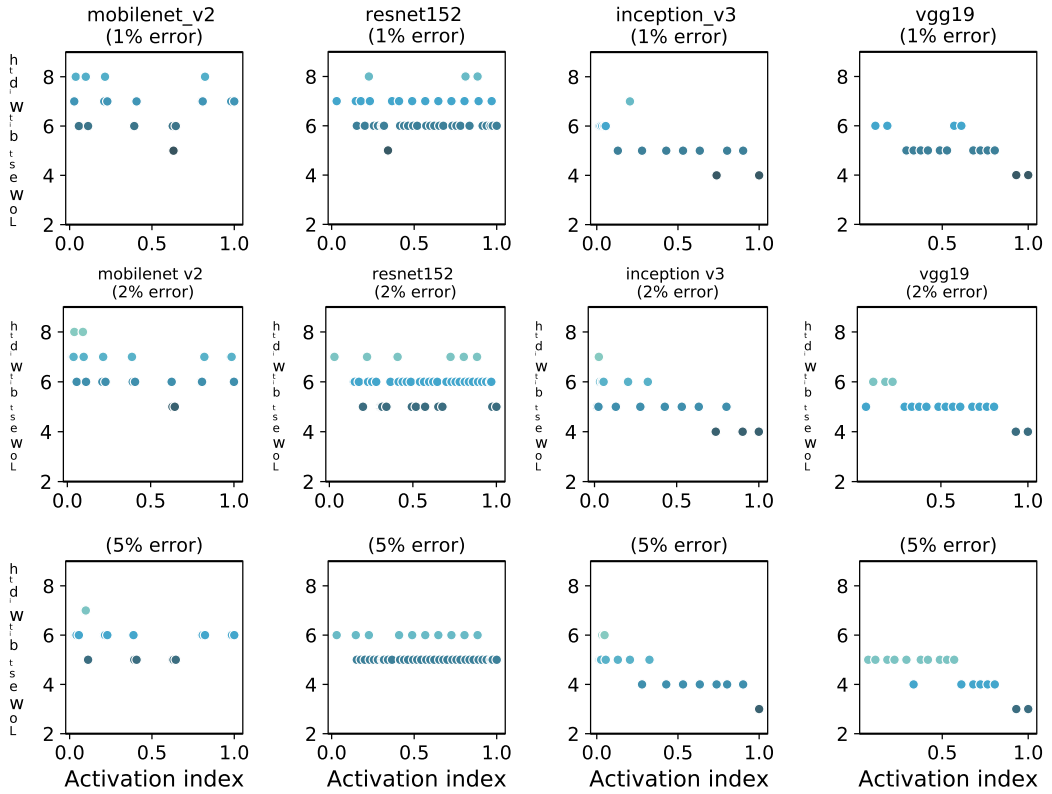


图 8. 每层最佳 ISQuant 位宽，精度下降最大为 1、2 和 5 pp。层索引通过每个 CNN 的层数进行归一化。大多数 CNN 可以量化为 3-8 位，且精度下降最小。

CNN-ISPM (图 6)。ReLU 输出分布在 CNN 的整个深度，始终保持最高的压缩比，VGG 19 高达 60 $\times$ 。MobileNetV2 和 Inception 模块的趋势类似，以一个或多个 ReLU 结束。

要点：ReLU 等激活广泛分布在流行的 CNN 中，它们的输出是压缩的重要候选者，在使用特定于输入的量化、位混洗和 LZ4 时，可以以最小的开销显著减小大小。

4.2.2 精度对压缩的敏感性。除了打包可能节省的空间之外，DynO 还需要估计其对准准确性的影响，以便更好地分区和安排执行。为了实现这一目标，我们将 CNN-ISPM 应用于每个可能的分割的依赖关系，并测量对 ImageNet 验证准确性的影响。由于 DynO 在技术上允许在任何层对图进行分区，因此对于较大的网络来说，此过程可能会变得非常耗时。在我们之前的结果的推动下，我们将分区搜索空间减少到仅具有 ReLU 依赖项的层。平均而言，这种启发式方法在所检查的 CNN 中将搜索空间减少了 81%，从而大大减少了离线分析和调度程序运行时间。我们的打包算法仍然适用于其他激活，但会产生不同的压缩动态。例如，Swish 函数将显示从较小范围的值到 $-\infty$ 的增益，而不是 ReLU 的硬零界限。

图 8 显示了 ISQuant 在精度降低 1、2 和 5 个百分点 (pp) 的情况下可实现的最低位宽。行代表退化允许水平，列代表不同的 CNN。

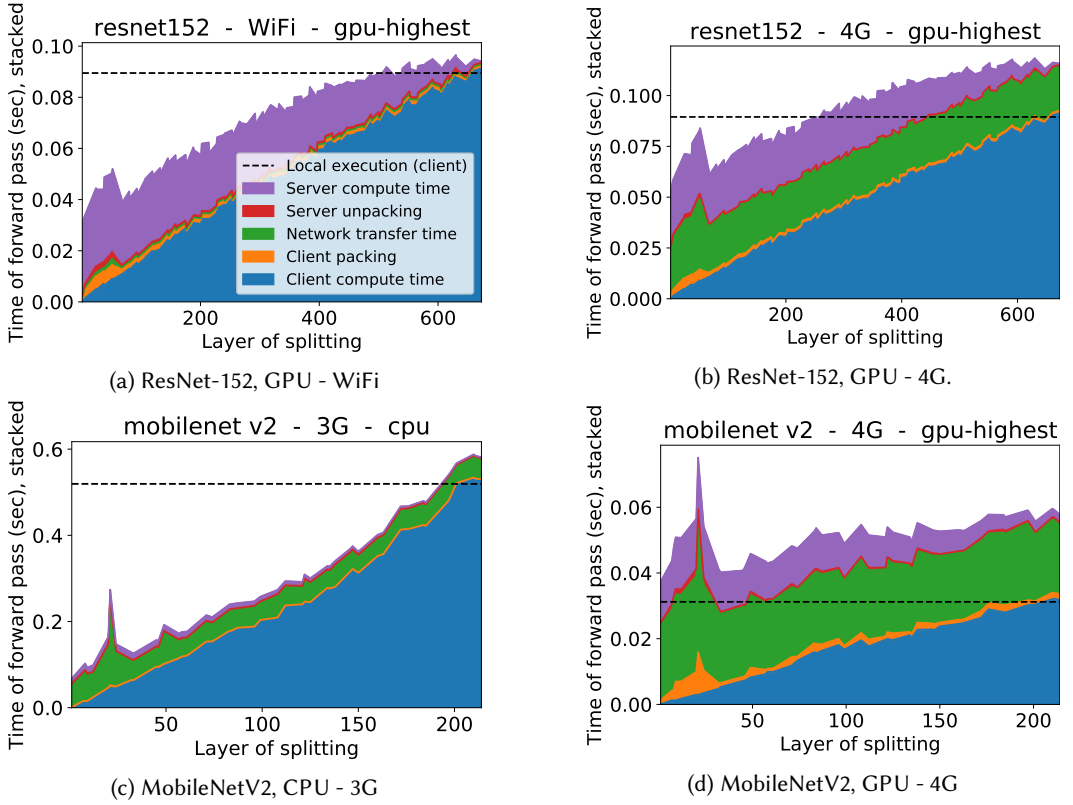


Fig. 9. Breakdown of the runtime and overheads for splitting a CNN at various layers for four very diverse scenarios. For both gpu-highest and cpu, we use the AGX's MAXN mode.

As it can be observed, layers' resilience to quantization varies from 3 to 8 bits across networks. For Inception-v3 and VGG19, we also observe higher tolerance to quantization deeper in the CNN. Compared to the status-quo 32-bit floats and typical 8-bit quantized precision, we are able to achieve 4-10.6 $\times$ and 1-2.6 $\times$ smaller size respectively with quantization only. Combining 4-bit ISQuant with the three-staged CNN-ISPM packing mechanism can achieve up to 60 $\times$ compression. A 5-pp drop can go as low as 3 bits. Even for critical applications that cannot tolerate drops above 1 pp, there are still many activations that can be quantized down to 5-6 bits, with only three layers needing higher precision across all CNNs.

Takeaways: *Different layers have different precision requirements for the same accuracy drop allowance. Split points with ReLUs as dependencies are good candidates, able to be quantized down to 4 bits with under 1-pp accuracy drop. This is important as transfer time is a major deterrent when shipping computation to a remote end which generic offloading systems do not explicitly optimize.*

4.3 Performance Analysis

This section assesses the quality of DynO's (split, packing) decisions and the achieved performance on diverse CNNs in a broad range of device capabilities and network conditions.

4.3.1 Split decisions and computation time. Fig. 9 breaks down the DynO's runtime for four diverse scenarios. For each possible split point (x-axis), we used DynO to run each CNN with the corresponding splitting and measured the runtime breakdown (y-axis). Furthermore, for each split

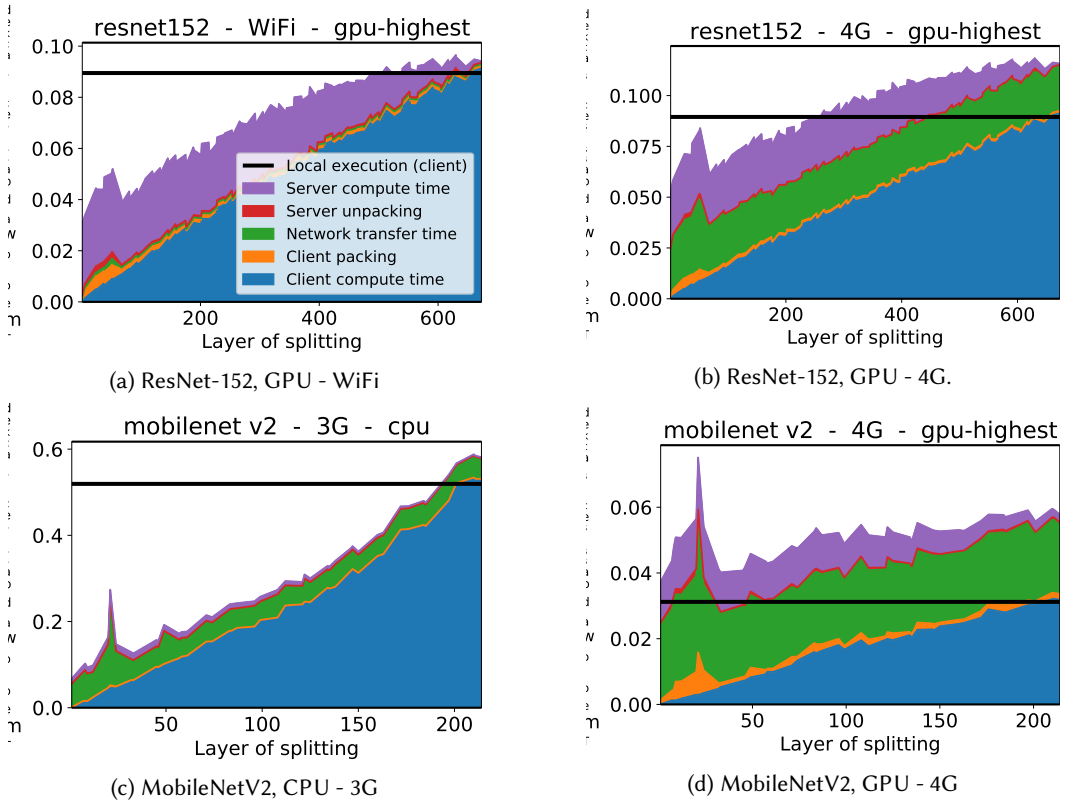


图 9. 针对四种截然不同的场景在不同层分割 CNN 的运行时间和开销的细分。对于 gpu-highest 和 cpu，我们都使用 AGX 的 MAXN 模式。

可以看出，网络中层对量化的弹性从 3 位到 8 位不等。对于 Inception-v3 和 VGG19，我们还观察到 CNN 更深层次的量化容忍度更高。与现状的 32 位浮点数和典型的 8 位量化精度相比，仅通过量化，我们就能分别实现小 4-10.6 $\times$ 和 1-2.6 $\times$ 的尺寸。将 4 位 ISQuant 与三级 CNN-ISPMM 打包机制相结合，可以实现高达 60 $\times$ 的压缩。5-pp 下降可以低至 3 位。即使对于不能容忍超过 1 pp 的下降的关键应用，仍然有许多激活可以量化到 5-6 位，所有 CNN 中只有三层需要更高的精度。

要点：对于相同的精度落差余量，不同的层有不同的精度要求。以 ReLU 作为依赖项的分割点是很好的候选者，能够量化到 4 位，且精度下降低于 1-pp。这很重要，因为将计算传输到通用卸载系统未明确优化的远程端时，传输时间是一个主要阻碍因素。

4.3 性能分析

本节评估 DynO 的(分割,打包)决策的质量以及在各种设备功能和网络条件下在不同 CNN 上实现的性能。

4.3.1 分割决策和计算时间。图 9 细分了 DynO 在四种不同场景下的运行时间。对于每个可能的分割点 (x 轴)，我们使用 DynO 运行具有相应分割的每个 CNN，并测量运行时间细分 (y 轴)。此外，对于每个分割

point we configured CNN-ISPM with the shortest bitwidth that led to less than 1-pp drop in accuracy.

In Fig. 9a, a large-scale CNN (ResNet-152) is run on a powerful client GPU (30W Jetson) under good network conditions (WiFi). Despite the powerful client, the fast, reliable channel results in remote execution yielding lower latency, due to the low transfer time. The partition point of zero signifies the server-only execution, while the device-only execution is explicitly shown by the horizontal dashed line. For all partition points in between (35-90 ms), there are savings in cloud usage, by progressively pushing more computation towards the device's spare resources.

Fig. 9b shows the same scenario over 4G. The significantly higher transmission overhead in 4G makes remote execution less favorable, especially after the device executes the first 250 layers. Moreover, we can see that there is a bigger impact from layers producing more information. Nonetheless, we note that remote execution would be unfeasible without CNN-ISPM and highlight that, for both Fig. 9a and 9b, when the server and client times are balanced (e.g. layer 150 in Fig. 9a), DynO can double the throughput through pipelining.

Fig. 9c depicts a different scenario that represents most mid-tier smartphones, where inference runs on CPUs. To realistically capture this scenario, we ran a lightweight CNN (MobileNetV2). Despite the mobile-friendly CNN and even when 3G is used, the remote assistance is almost always a better option. In contrast, when a more powerful compute engine is employed (GPU in Fig. 9d), the faster local inference time can lead DynO's scheduler to perform full onloading and execute locally. Across all cases, the overhead of CNN-ISPM ("client packing" in Fig. 9) is relatively small compared to the total CNN computation times.

Takeaways: *The target CNN's workload, the client/server capabilities and the network conditions can result in different execution dynamics. For most settings, there are opportunities to onload computation to the client to optimize for cloud costs, throughput or latency. This also demonstrates why a dynamic scheduler can play a key role in coping with such dynamic and heterogeneous environments.*

4.3.2 DynO inference throughput. In this section, we assess DynO's attainable performance in throughput-driven cases. To this end, we configured DynO to use pipelining and its scheduler to maximize inference throughput with an accuracy drop tolerance of ≤ 1 pp. In Fig. 10, we scale the client CNN computation and packing times by a given factor as a proxy of different client capabilities. Different slowdown factors along the x-axis can also be interpreted as fluctuating on-device load.

For all CNNs, splitting the network can result in significant throughput gains compared to full-remote execution. For ResNet-152 (Fig. 10a), DynO always yields at least equal or higher throughput compared to both fully-local and remote executions. For smaller CNNs such as MobileNetV2 (Fig. 10b), DynO achieves significant speedups of up to 10.5 $\times$ and 35.5 $\times$ over full-server and client execution, respectively. On the other hand, VGG19 (Fig. 10d) is a notable example of a CNN with significant transmission overhead due to its large tensor sizes, especially under cellular networks. For such CNNs the scheduler defaults to local execution, even for devices that are 5 $\times$ slower than Jetson. Nonetheless, when targeting even more resource-constrained devices ($> 5\times$), DynO outperforms local execution by up to 4.2 $\times$ for 4G and 1.8 $\times$ for 3G.

Overall, for faster devices (i.e. left of the x-axis in Fig. 10a-10d), DynO adopts a more strict onloading policy by assigning more workload to the client. As the device becomes busier (e.g. increased load from concurrent apps) or a lower-end platform is targeted (to the right of the x-axis), DynO applies onloading more conservatively and the computation is progressively assisted by the server.

Takeaways: *In most cases, it is possible to find a split where pipelining improves throughput, with significant gains over local and remote execution. In general, the faster the client, the more justified it is to onload computation there. Only in scenarios with very slow network, huge CNNs or low-end*

我们将 CNN-ISPm 配置为最短的位宽，导致精度下降不到 1-pp。

在图9a中，大规模CNN（ResNet-152）在良好的网络条件（WiFi）下在强大的客户端GPU（30W Jetson）上运行。尽管客户端功能强大，但由于传输时间短，快速、可靠的通道可以实现远程执行，从而降低延迟。分区点零表示仅服务器执行，而仅设备执行由水平虚线明确显示。对于介于 (35-90 ms) 之间的所有分区点，通过逐步将更多计算推向设备的备用资源，可以节省云使用量。

图 9b 显示了 4G 上的相同场景。4G 中显着较高的传输开销使得远程执行不太有利，尤其是在设备执行前 250 层之后。此外，我们可以看到产生更多信息的层会产生更大的影响。尽管如此，我们注意到，如果没有 CNN-ISPm，远程执行将是不可行的，并强调，对于图 9a 和 9b，当服务器和客户端时间平衡时（例如图 9a 中的第 150 层），DynO 可以通过管道使吞吐量加倍。

图 9c 描绘了代表大多数中型智能手机的不同场景，其中推理在 CPU 上运行。为了真实地捕捉这种场景，我们运行了轻量级 CNN (MobileNetV2)。尽管 CNN 适合移动设备，甚至在使用 3G 时，远程协助几乎总是更好的选择。相比之下，当采用更强大的计算引擎（图 9 d 中的 GPU）时，更快的本地推理时间可以导致 DynO 的调度程序执行完整加载并在本地执行。在所有情况下，与 CNN 总计算时间相比，CNN-ISPm 的开销（图 9 中的“客户端打包”）相对较小。

要点：目标 CNN 的工作负载、客户端/服务器功能和网络条件可能会导致不同的执行动态。对于大多数设置，有机会将计算加载到客户端以优化云成本、吞吐量或延迟。这也说明了为什么动态调度程序可以在应对这种动态和异构环境中发挥关键作用。

4.3.2 DynO 推理吞吐量。在本节中，我们评估 DynO 在吞吐量驱动的情况下可达到的性能。为此，我们将 DynO 配置为使用管道及其调度程序，以 ≤ 1 pp 的精度下降容差最大化推理吞吐量。在图 10 中，我们按给定因子缩放客户端 CNN 计算和打包时间，作为不同客户端功能的代理。沿 x 轴的不同减速因素也可以解释为设备负载的波动。

对于所有 CNN，与完全远程执行相比，分割网络可以带来显着的吞吐量增益。对于 ResNet-152（图 10a），与完全本地和远程执行相比，DynO 始终产生至少相同或更高的吞吐量。对于 MobileNetV2（图 10b）等较小的 CNN，DynO 比全服务器和客户端执行分别实现了高达 10.5 $\times$ 和 35.5 $\times$ 的显着加速。另一方面，VGG19（图 10d）是 CNN 的一个著名示例，由于其张量大小较大，因此具有显着的传输开销，尤其是在蜂窝网络下。对于此类 CNN，调度程序默认为本地执行，即使对于比 Jetson 慢 5 $\times$ 的设备也是如此。尽管如此，当针对资源更加受限的设备 ($> 5\times$) 时，DynO 的性能比本地执行高出 4.2 $\times$ (4G) 和 1.8 $\times$ (3G)。

总体而言，对于更快的设备（即图 10a-10d 中 x 轴的左侧），DynO 通过向客户端分配更多工作负载来采用更严格的加载策略。随着设备变得更加繁忙（例如，并发应用程序的负载增加）或目标为低端平台（x 轴右侧），DynO 会更加保守地应用加载，并且服务器逐渐协助计算。

要点：在大多数情况下，可以找到一个分割点，其中流水线可以提高吞吐量，并且比本地和远程执行有显着的收益。一般来说，客户端越快，在那里加载计算就越合理。仅适用于网络非常慢、CNN 巨大或低端的场景

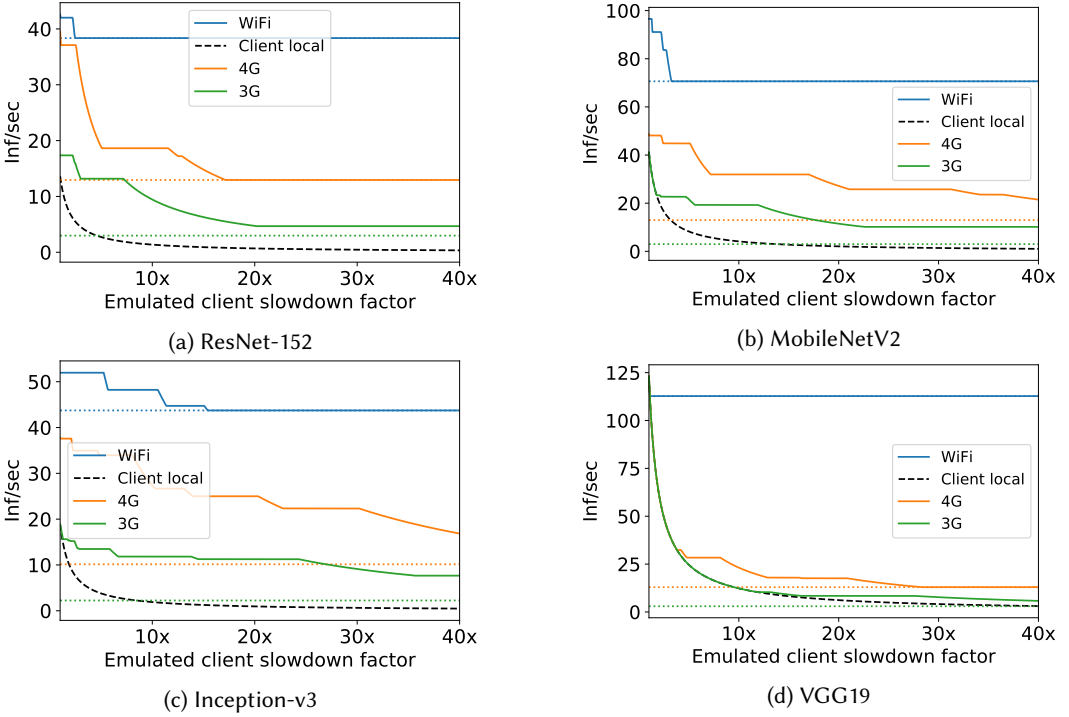


Fig. 10. Throughput achieved for varying network conditions and device capabilities/load. Dotted lines represent full-server execution (using CNN-ISP on the CNN’s inputs). Dashed black lines represent local-only execution. For reference, flagship mobile GPUs/CPU’s are typically 5×/20× slower than Jetson’s GPU, respectively, while low-tier devices around 35× slower.

clients, does the behavior default to one of the two extremes. These diverse decisions observed under varying device conditions highlight the need for joint decisions on the onloading and transmission policies at runtime.

4.3.3 DynO server savings vs. performance SLOs. Here, we assess DynO’s multi-objective scheduler’s ability to capture novel complex scenarios such as saving cloud cost under strict performance constraints and diverse network conditions. Fig. 11 shows the attainable server savings when a certain latency deadline is requested. For ResNet-152 (Fig. 11a-11b), DynO onloads the whole computation when the deadlines are lenient ($>2s$ per inference) even for slower clients (27× slower than 30W Jetson). For tighter latency deadlines, server assistance is required. However, across all deadline goals DynO can *gradually* onload computation based on the device capabilities, saving cloud costs. For instance, for a 500-ms deadline in Fig. 11a, DynO onloads the whole network for high-tier devices, and close to 20% for low-tier devices. Similar flexibility is shown over 4G (Fig. 11b), although stricter deadlines of 50 ms and 100 ms cannot always be met for slower devices. Lighter networks such as MobileNetV2 (Fig. 11c-11d) show similar performance, but achieve higher server savings as a larger part can be executed on-device.

Takeaways: *Given the heterogeneous device landscape, we can automatically optimize how much is unloaded to each device, based on the CNN, device capabilities and load, as well as network conditions, resulting in significant savings and improved inference performance.*

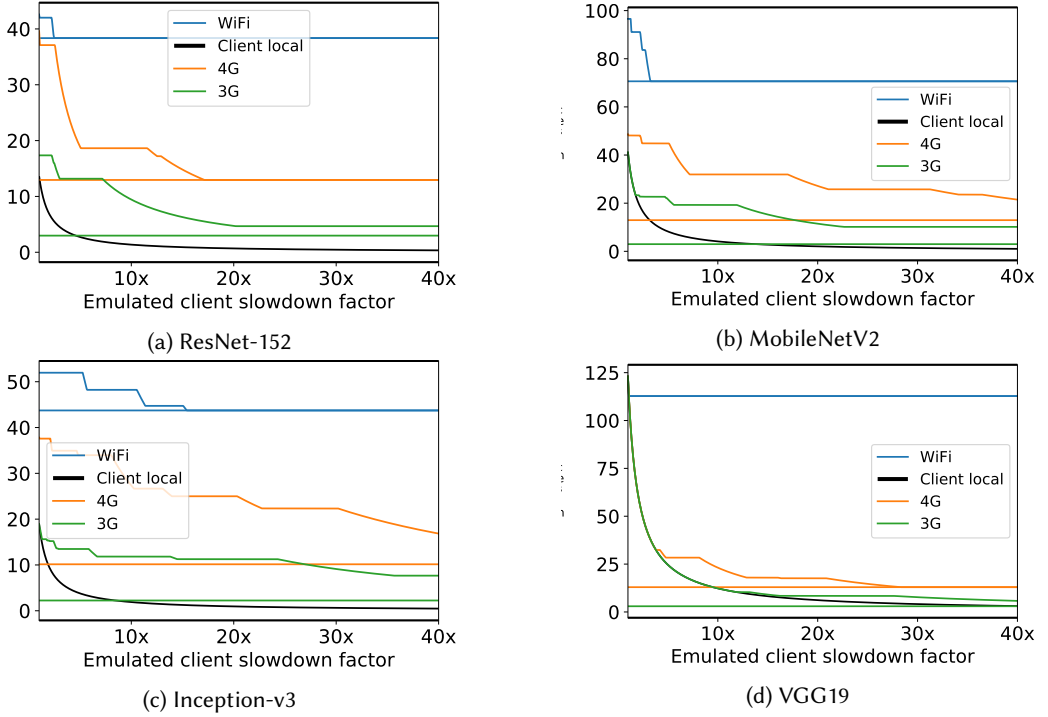


图 10. 在不同的网络条件和设备功能/负载下实现的吞吐量。虚线代表全服务器执行（在 CNN 的输入上使用 CNN-ISP）。黑色虚线表示仅本地执行。作为参考，旗舰移动 GPU/CPU 通常分别比 Jetson 的 GPU 慢 5x/20x，而低端设备则慢 35x 左右。

客户，行为是否默认为两个极端之一。在不同的设备条件下观察到的这些不同的决策凸显了在运行时对加载和传输策略进行联合决策的必要性。

4.3.3 DynO 服务器节省与性能 SLO。在这里，我们评估了 DynO 的多目标调度器捕获新颖的复杂场景的能力，例如在严格的性能约束和不同的网络条件下节省云成本。图 11 显示了当请求特定延迟期限时可实现的服务器节省。对于 ResNet-152（图 11a-11b），即使对于速度较慢的客户端（比 30W Jetson 慢 27x），DynO 在截止时间宽松时（每个推理 2 秒）也会加载整个计算。对于更严格的延迟期限，需要服务器协助。然而，在所有截止日期目标中，DynO 可以根据设备功能逐步加载计算，从而节省云成本。例如，对于图 11a 中的 500 毫秒截止时间，DynO 为高层设备加载整个网络，为低层设备加载接近 20%。4G 也显示出类似的灵活性（图 11b），尽管对于速度较慢的设备而言，并不总能满足更严格的 50 毫秒和 100 毫秒的期限。MobileNetV2（图 11c-11d）等较轻的网络表现出类似的性能，但由于较大的部分可以在设备上执行，因此节省了更多的服务器资源。

要点：考虑到异构设备环境，我们可以根据 CNN、设备功能和负载以及网络条件自动优化每个设备的加载量，从而显著节省成本并提高推理性能。

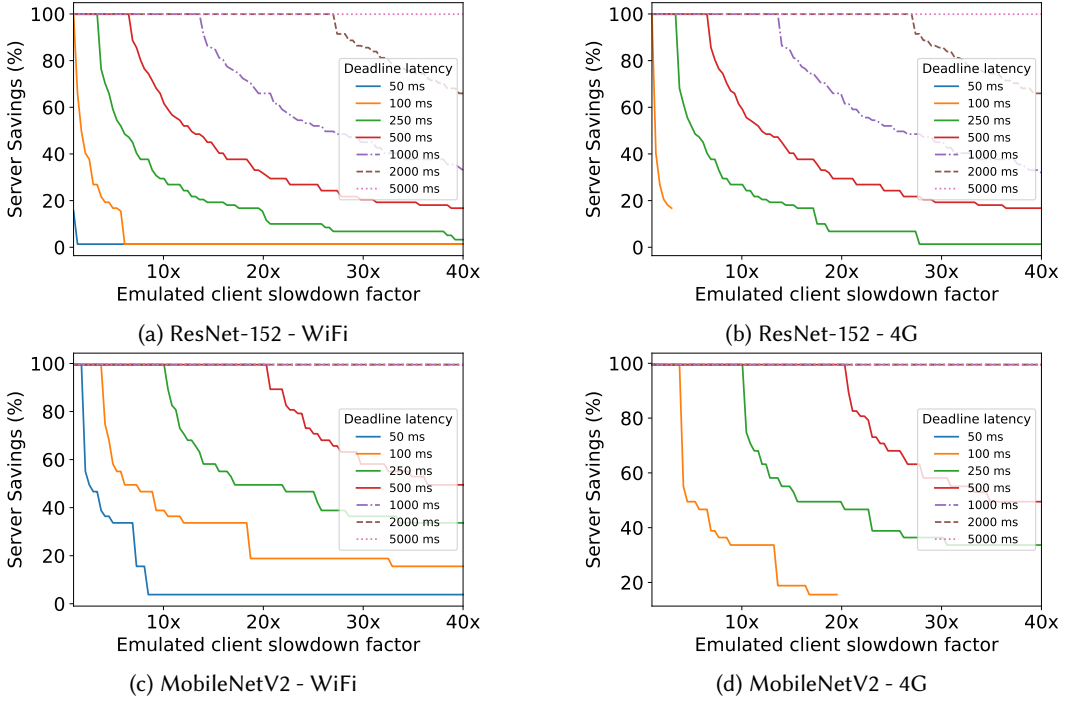


Fig. 11. Percentage of server computation saved for varying device capabilities to meet a latency deadline. When a line stops (e.g. (b) 100 ms latency), the requested deadline cannot be met.

4.3.4 Comparison with existing frameworks. In this section, we evaluate DynO against the state-of-the-art CNN offloading system, Neurosurgeon (Section 2), and the status-quo server- and device-only baselines. For the server-only baseline, we employed an optimized variant that is enhanced with CNN-ISPM’s compression to pack the inputs prior to transmission. DynO’s scheduler was configured to maximize throughput with an 1-pp of maximum accuracy drop tolerance. Similarly, Neurosurgeon was implemented with latency minimization as its objective. Fig. 12 shows the achieved throughput for varying network conditions and devices. We can see that server-only execution is communication-bound, with its throughput following the trajectory of the available bandwidth. In contrast, Neurosurgeon can tunably distribute computation between device and server. However, its scheduler provides polarized partitioning, switching from “offload-nothing” to “offload-everything”. The gradual performance increase deceptively resembles progressive offloading, but depicts in fact the increase of bandwidth when Neurosurgeon selects *full-offloading*. Compared to the enhanced server-only baseline, Neurosurgeon has a lower peak throughput due to CNN-ISPM’s compression that the former incorporates.

DynO achieves the highest throughput across all setups. First, we observe that for powerful devices (Jetson-30W in Fig. 12a-12b) and lower network speeds (<1Mbps), distributed execution cannot surpass the throughput of client-only execution and hence DynO selects on-device execution. This manifests because the cost of packing and transferring the data outweighs the benefits of server assistance. On the other hand, for lower-end devices (e.g. Fig. 12c-12d), distributed execution yields speedups even under scarce network conditions. As networking improves (to the right of Fig. 12c), DynO’s pipelining provides a further boost with up to 10× higher throughput than local execution at 10Mbps, before its scheduler switches to full offloading. Finally, DynO yields equal or

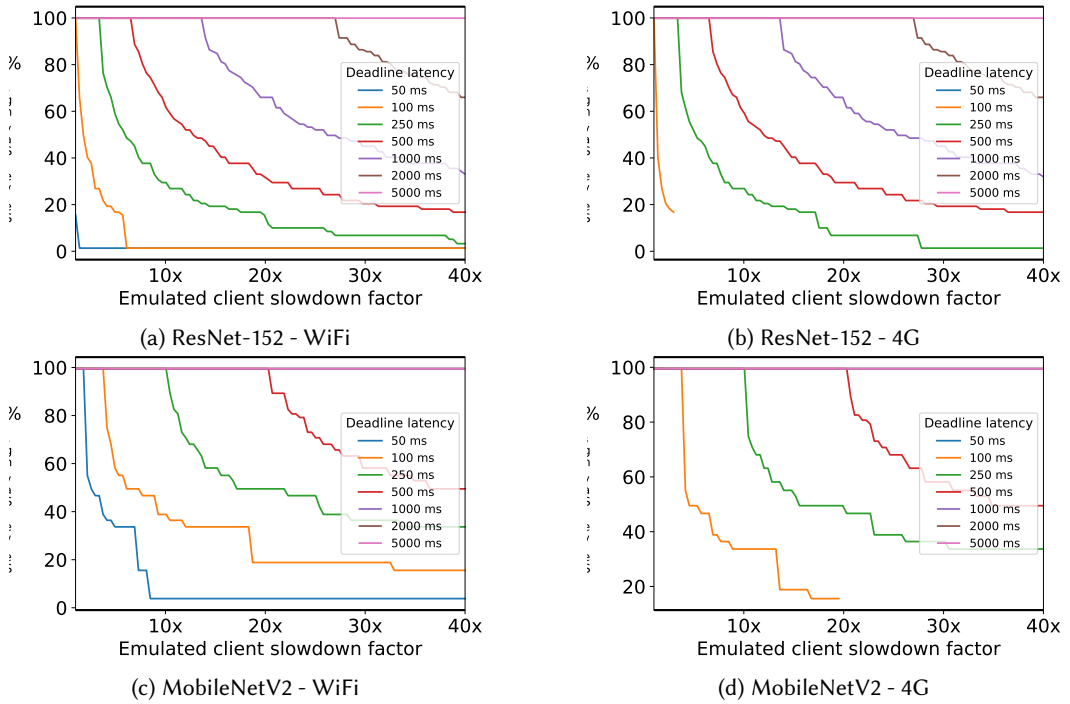


图 11. 为满足延迟期限而针对不同设备功能节省的服务器计算百分比。当线路停止时（例如 (b) 100 毫秒延迟），无法满足请求的截止时间。

4.3.4 与现有框架的比较。在本节中，我们将根据最先进的 CNN 卸载系统、神经外科医生（第 2 节）以及仅用于服务器和设备的现状基线来评估 DynO。对于仅服务器基线，我们采用了一种优化的变体，该变体通过 CNN-ISPM 的压缩进行了增强，以在传输之前打包输入。DynO 的调度程序配置为最大程度地提高吞吐量，并具有 1-pp 的最大精度跌落容差。同样，神经外科医生的实施也以最小化延迟为目标。图 12 显示了不同网络条件和设备所实现的吞吐量。我们可以看到，仅服务器执行是受通信限制的，其吞吐量遵循可用带宽的轨迹。相比之下，神经外科医生可以在设备和服务器之间调整分配计算。然而，它的调度程序提供了极化分区，从“卸载无”切换到“卸载所有”。逐渐的性能提高看似类似于渐进卸载，但实际上描述了当神经外科医生选择完全卸载时带宽的增加。与增强型仅服务器基线相比，由于前者采用了 CNN-ISPM 压缩，Neurosurgeon 的峰值吞吐量较低。

DynO 在所有设置中实现了最高吞吐量。首先，我们观察到，对于功能强大的设备（图 12a-12b 中的 Jetson-30W）和较低的网络速度（<1Mbps），分布式执行无法超过仅客户端执行的吞吐量，因此 DynO 选择设备上执行。这是因为打包和传输数据的成本超过了服务器协助的好处。另一方面，对于低端设备（例如图 12c-12d），即使在稀缺的网络条件下，分布式执行也会产生加速。随着网络的改进（图 12c 右侧），在调度程序切换到完全卸载之前，DynO 的流水线技术提供了进一步的提升，吞吐量比 10Mbps 的本地执行高出 10x。最后，DynO 产生等于或

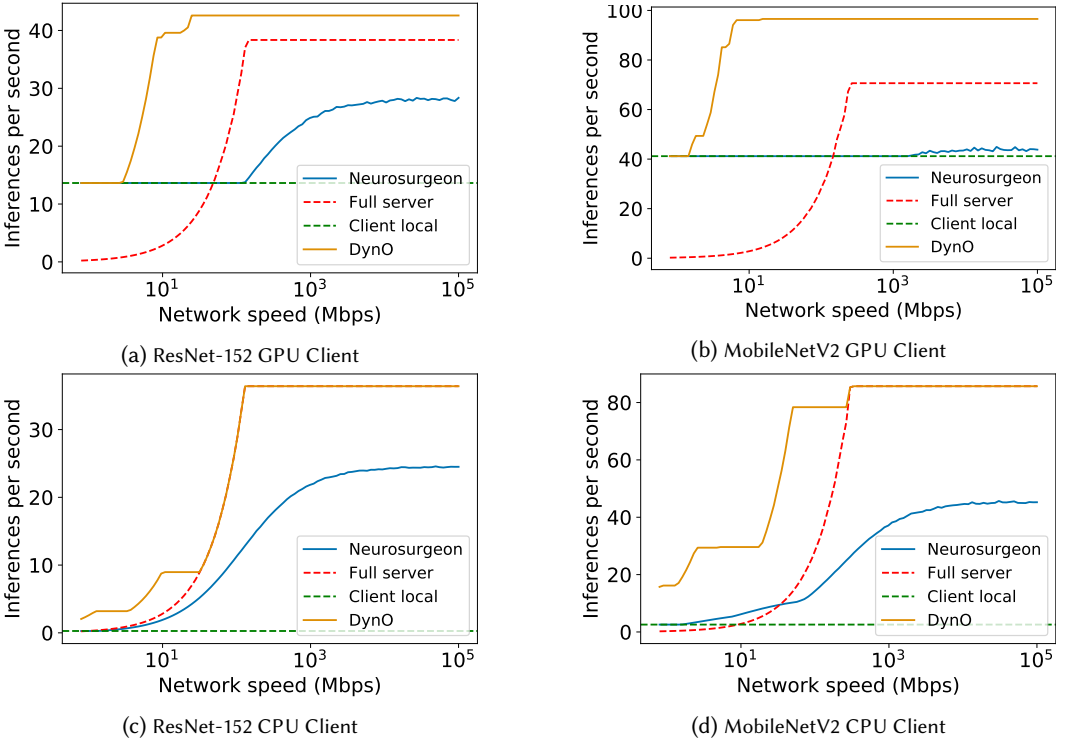


Fig. 12. Distributed throughput with pipelining.

higher peak inference rate than the “full-server” paradigm, attributed to the smaller transfer size due to the additional ISQuant step of CNN-ISPM as well as the strategic adaptive selection of split point.

Compared to Neurosurgeon, DynO consistently delivers higher throughput across all settings. On more compute-capable devices (Fig. 12a-12b), Neurosurgeon selects device-only execution for a significant portion of bandwidths. This is due to the fast processing of the client and the high communication cost. In contrast, DynO’s CNN-ISPM mechanism significantly reduces the volume of transferred data and effectively alleviates the communication overhead. Hence, while Neurosurgeon requires high bandwidths to switch from on-device execution (>20 Mbps for ResNet-152 and >100 Mbps for MobileNetV2), DynO significantly reduces the bandwidth needs and makes distributed execution feasible below 1Mbps. On lower-tier devices (Fig. 12c-12d), although Neurosurgeon switches to offloading from earlier on due to the slower client, DynO is still able to exploit distributed execution from very low bandwidths and with higher throughput (up to 6.5 $\times$) due to the faster packed transmission of dependencies. Moreover, under abundant bandwidth (to the right of the x-axis), the impact of DynO’s pipelining dominates, leading to speedup of up to 7.9 $\times$.

Takeaways: *The transfer size of dependencies when onloading plays a critical role to the feasible splits of a CNN, especially under bandwidth-constrained channels. As such, the co-optimization of both the selected split point and data packing strategy leads to throughput gains not previously attainable. Moreover, in streaming scenarios, pipelining offers an additional boost in the system’s throughput.*

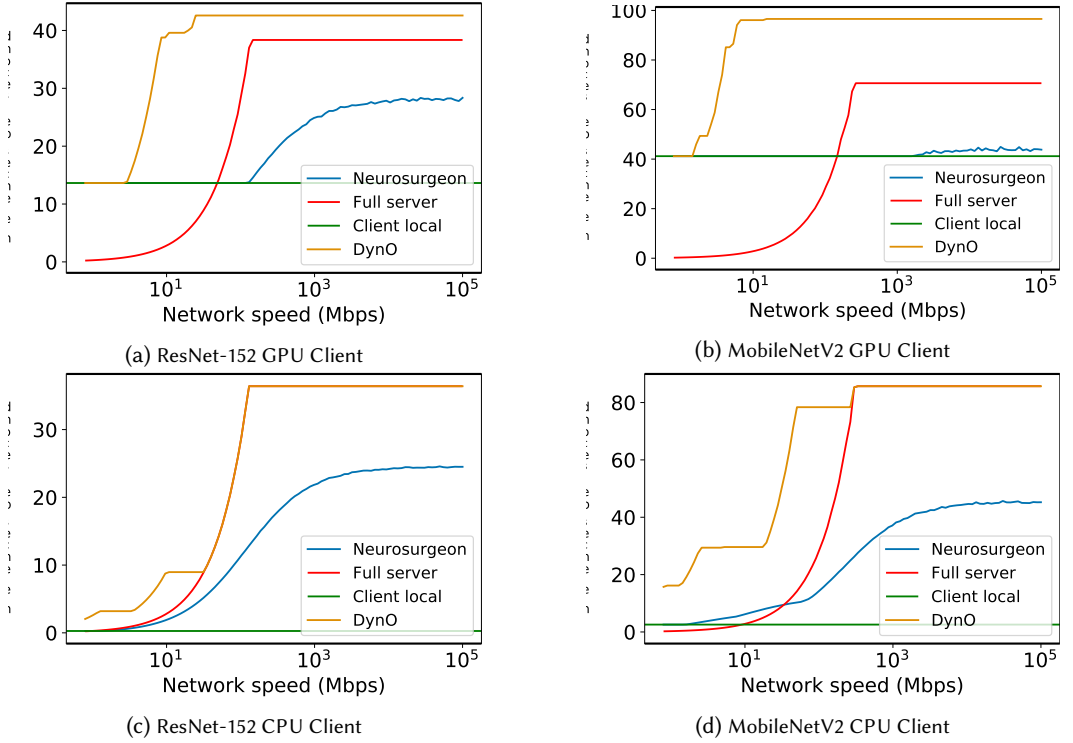


图 12. 流水线的分布式吞吐量。

与“全服务器”范式相比，峰值推理率更高，这归因于 CNN-ISPM 的额外 ISQuant 步骤以及分割点的策略自适应选择带来的更小的传输大小。

与神经外科医生相比，DynO 在所有设置中始终提供更高的吞吐量。在更多具有计算能力的设备上（图 12a-12b），神经外科医生为很大一部分带宽选择仅设备执行。这是由于客户端的快速处理和较高的通信成本。相比之下，DynO 的 CNN-ISPM 机制显著减少了传输的数据量，有效减轻了通信开销。因此，虽然 Neurosurgeon 需要高带宽才能从设备上执行切换（ResNet-152 为 >20Mbps，MobileNetV2 为 >100Mbps），但 DynO 显著降低了带宽需求，并使分布式执行速度低于 1Mbps。在较低层设备上（图 12c-12d），尽管 Neurosurgeon 由于客户端速度较慢而从早期切换到卸载，但由于依赖项的打包传输速度更快，DynO 仍然能够利用非常低的带宽和更高的吞吐量（高达 6.5 $\times$ ）的分布式执行。此外，在充足的带宽（x 轴右侧）下，DynO 流水线的影响占主导地位，导致加速高达 7.9 $\times$ 。

要点：加载时依赖项的传输大小对于 CNN 的可行分割起着至关重要的作用，尤其是在带宽受限的通道下。因此，所选分割点和数据打包策略的共同优化会带来以前无法实现的吞吐量增益。此外，在流场景中，流水线可以进一步提高系统的吞吐量。

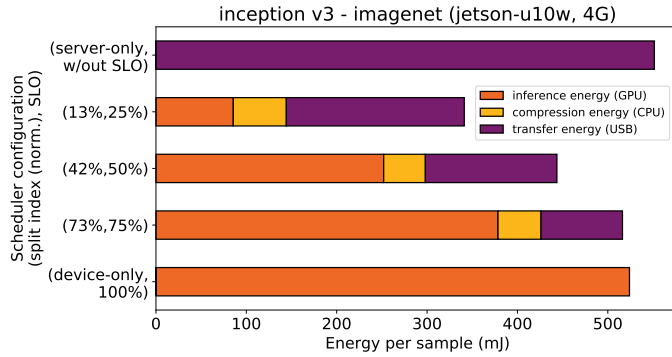


Fig. 13. Energy consumption of DynO vs. baselines. SLOs are expressed as percentages of the device-only latency.

4.4 Energy Consumption

In this experiment, we quantify the energy consumption of DynO’s components in different deployment scenarios compared against the *server-only* and *device-only* baselines. Computation energy (CPU, GPU) was measured from the OS probes on Jetson Xavier (underclocked 10W), while the transmission energy was quantified over (Anonymous Provider’s) 4G network with a Huawei E3372 USB adapter connected to a Monsoon AAA10F power monitor. We used the 10-watt mode of Jetson Xavier to better represent the current power-constrained mobile platforms.

Fig. 13 depicts the energy breakdown per component, measured over 100 inferences of Inception-v3. From top to bottom, the first bar represents the case where no onloading occurs, *i.e.* the client transfers the JPEG compressed data to the server for inference. This yields the highest energy consumption, at 524 mJ per inference, due to the size of the transferred sample. For the next three bars, we vary the SLO – expressed as the percentage of max allowed latency compared to device-only inference – which in turn leads DynO to select different $\langle$ split, packing $\rangle$ parameters. We witness lower overall energy consumption per sample, ranging from 61.8% to 93.6% of the server-only, a fact that we attribute to the significant impact of CNN-ISP’s packing. Last, the bottom bar depicts the device-only inference, which yields 95% of the server-only energy.

Takeaways: *Even without explicitly optimizing for it, there can be energy gains from onloaded execution from the partial on-device computation and compressed transmission.*

5 LIMITATIONS & FUTURE WORK

While DynO supports a wide range of features and DNN architectures, there are limitations in our study. First, we only consider onloading between two devices and assume models already reside in both ends. Model distribution [29] and client multi-tenancy [11] on the server side are interesting adjacent issues, but we do not tackle them in this study.

In terms of our evaluation, we varied the network conditions through traffic shaping to assess our scheduler behavior. Moreover, we did not evaluate on heterogeneous devices, but rather targeted the highly configurable Jetson AGX, which enabled us to use it as a proxy for a wide range of device tiers by performing dynamic voltage and frequency scaling and linearly scaling the measured latency. Last, we focus our paper on widely deployed CNNs which largely employ ReLU activations. Different networks, such as RNNs or models with different activation functions (*e.g.* Swish [51] or PReLU [19]), could also have compression potential under onloading. Nonetheless, DynO can still be beneficial for more generic, non-sparsity-inducing split points, as shown in Fig. 6.

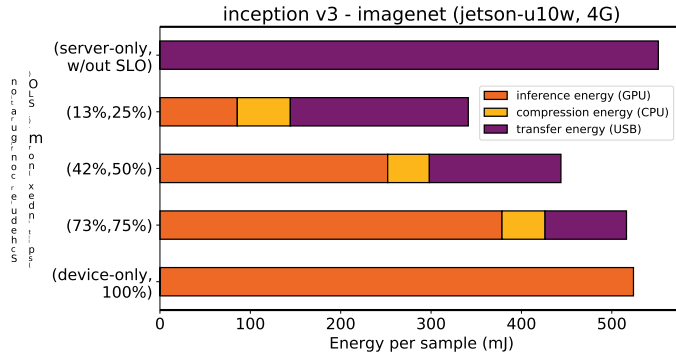


图 13. DynO 的能耗与基线的比较。SLO 以仅设备延迟的百分比表示。

4.4 能源消耗

在本实验中，我们与仅服务器和仅设备基线相比，量化了不同部署场景中 DynO 组件的能耗。计算能量（CPU、GPU）是通过 Jetson Xavier（降频 10W）上的操作系统探针测量的，而传输能量是通过连接到 Monsoon AAA10F 功率监视器的华为 E3372 USB 适配器通过（匿名提供商）4G 网络进行量化的。我们使用 Jetson Xavier 的 10 瓦模式来更好地代表当前功耗受限的移动平台。

图 13 描述了每个组件的能量分解，通过 Inception-v3 的 100 个推论进行测量。从上到下，第一条表示没有发生加载的情况，即客户端将 JPEG 压缩数据传输到服务器进行推理。由于传输样本的大小，这会产生最高的能耗，每次推理 524 mJ。对于接下来的三个条形图，我们改变了 SLO（表示为与仅设备推理相比最大允许延迟的百分比），这反过来又导致 DynO 选择不同的 $\langle \text{split, Packing} \rangle$ 参数。我们发现每个样本的总体能耗较低，从仅服务器的 61.8% 到 93.6%，我们将这一事实归因于 CNN-ISPM 打包的显着影响。最后，底部栏描述了仅设备的推理，它产生了仅服务器能量的 95%。

要点：即使没有明确对其进行优化，部分设备上计算和压缩传输的加载执行也可以带来能量增益。

5 局限性和未来的工作

虽然 DynO 支持广泛的功能和 DNN 架构，但我们的研究存在局限性。首先，我们只考虑两个设备之间的加载，并假设模型已经驻留在两端。服务器端的模型分发[29]和客户端多租户[11]是有趣的相邻问题，但我们在本研究中不解决它们。

在我们的评估方面，我们通过流量整形改变网络条件来评估我们的调度程序行为。此外，我们没有在异构设备上进行评估，而是针对高度可配置的 Jetson AGX，这使我们能够通过执行动态电压和频率缩放以及线性缩放测量的延迟，将其用作各种设备层的代理。最后，我们的论文重点关注广泛部署的 CNN，它们主要采用 ReLU 激活。不同的网络，例如 RNN 或具有不同激活函数的模型（例如 Swish [51] 或 PReLU [19]），在加载下也可能具有压缩潜力。尽管如此，DynO 对于更通用、非稀疏性诱导的分割点仍然有益，如图 6 所示。

With respect to MobileNetV2's execution on Xavier's CPU, we noticed a bottlenecked execution behaviour with PyTorch and NNPACK on Xavier's CPU. Results from our profiling with NSight Systems (v. 2021.5) show frequent context switching and underutilisation of cores, even under varying thread counts. This indicates a low computation-to-communication ratio, but would require further investigation.

Furthermore, RNN-based models, such as LSTMs [21] and GRUs [10], constitute a special case when it comes to onloading due to their recurrence property: the RNN's internal state constitutes an *inter-sample* dependency that is not present in CNNs. As such, if the state-update computation is moved from the device to the server, or vice versa, the current state also has to be transferred to the other party. Hence, although DynO's partitioning strategy generalizes across DNN architectures through its automatic identification of split-point dependencies, RNNs would require a dedicated treatment. In the future, we would like to extend our work on different architectures and use-cases, and showcase the generalizability of our proposed framework.

6 CONCLUSION

This paper presents DynO, a distributed CNN inference framework that seamlessly splits computation across device and cloud. By exploiting the variable precision requirements along a given CNN, the proposed system introduces an input-specific quantization method that tunably minimizes the data transfer overhead. At run time, DynO jointly tunes the splitting and data packing policy to tailor the execution to the use-case multi-objective needs. DynO delivers significant performance gains over state-of-the-art CNN offloading systems, while saving on cloud cost by onloading to the capable clients, without sacrificing energy efficiency.

REFERENCES

- [1] Mario Almeida, Stefanos Laskaridis, Ilias Leontiadis, Stylianos I. Venieris, and Nicholas D. Lane. 2019. EmBench: Quantifying Performance Variations of Deep Neural Networks Across Modern Commodity Devices. In *3rd International Workshop on Deep Learning for Mobile Systems and Applications (EMDL)*. ACM, 1–6.
- [2] Mario Almeida, Stefanos Laskaridis, Abhinav Mehrotra, Lukasz Dudziak, Ilias Leontiadis, and Nicholas D. Lane. 2021. Smart at What Cost? Characterising Mobile Deep Neural Networks in the Wild. In *Proceedings of the 21st ACM Internet Measurement Conference (Virtual Event) (IMC '21)*. Association for Computing Machinery, New York, NY, USA, 658–672. <https://doi.org/10.1145/3487552.3487863>
- [3] Mattia Antonini, Tran Huy Vu, Chulhong Min, Alessandro Montanari, Akhil Mathur, and Fahim Kawsar. 2019. Resource Characterisation of Personal-Scale Sensing Models on Edge Accelerators. In *Proceedings of the First International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things (AIChallengeIoT)*. ACM, 49–55.
- [4] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Gutttag. 2020. What is the State of Neural Network Pruning? In *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 2. 129–146.
- [5] J. Burgess. 2019. RTX ON – The NVIDIA TURING GPU. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. 1–27. <https://doi.org/10.1109/HOTCHIPS.2019.8875651>
- [6] Alejandro Cartas, Martin Kocour, Aravindh Raman, Ilias Leontiadis, Jordi Luque, Nishanth Sastry, Jose Nuñez-Martinez, Diego Perino, and Carlos Segura. 2019. A Reality Check on Inference at Mobile Networks Edge. In *Proceedings of the 2nd International Workshop on Edge Systems, Analytics and Networking (EdgeSys)*. 54–59.
- [7] Byung-Gon Chun, Sunghwan Ihm, Petros Maniatis, Mayur Naik, and Ashwin Patti. 2011. CloneCloud: Elastic Execution Between Mobile Device and Cloud. In *Proceedings of the Sixth Conference on Computer Systems (EuroSys '11)*. 301–314.
- [8] E. Chung et al. 2018. Serving DNNs in Real Time at Datacenter Scale with Project Brainwave. *IEEE Micro* 38, 2 (2018), 8–20.
- [9] Eduardo Cuervo, Aruna Balasubramanian, Dae-ki Cho, Alec Wolman, Stefan Saroiu, Ranveer Chandra, and Paramvir Bahl. 2010. MAUI: Making Smartphones Last Longer with Code Offload. In *International Conference on Mobile Systems, Applications, and Services (MobiSys)*.
- [10] Rahul Dey and Fathi M Salem. 2017. Gate-Variants of Gated Recurrent Unit (GRU) Neural Networks. In *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*. IEEE, 1597–1600.
- [11] Zhou Fang, Jeng-Hau Lin, Mani B. Srivastava, and Rajesh K. Gupta. 2019. Multi-Tenant Mobile Offloading Systems for Real-Time Computer Vision Applications. In *Proceedings of the 20th International Conference on Distributed Computing*

关于 MobileNetV2 在 Xavier CPU 上的执行, 我们注意到 PyTorch 和 NNPACK 在 Xavier CPU 上的执行行为出现瓶颈。我们使用 NSight Systems (v. 2021.5) 进行分析的结果显示, 即使在不同的线程数下, 上下文切换也频繁且核心利用率不足。这表明计算与通信的比率较低, 但需要进一步调查。

此外, 基于 RNN 的模型, 例如 LSTM [21] 和 GRU [10], 由于其递归特性而在加载方面构成了一种特殊情况: RNN 的内部状态构成了 CNN 中不存在的样本间依赖性。因此, 如果状态更新计算从设备移动到服务器, 或者反之亦然, 当前状态也必须传输到另一方。因此, 尽管 DynO 的分区策略通过自动识别分割点依赖关系在 DNN 架构中得到推广, 但 RNN 需要专门的处理。未来, 我们希望将我们的工作扩展到不同的架构和用例, 并展示我们提出的框架的通用性。

6 结论

本文介绍了 DynO, 一种分布式 CNN 推理框架, 可跨设备和云无缝分割计算。通过利用给定 CNN 的可变精度要求, 所提出的系统引入了一种特定于输入的量化方法, 可最大程度地减少数据传输开销。在运行时, DynO 联合调整分割和数据打包策略, 以根据用例多目标需求定制执行。与最先进的 CNN 卸载系统相比, DynO 提供了显著的性能提升, 同时通过卸载到有能力的客户端来节省云成本, 而无需牺牲能源效率。

参考

- [1] Mario Almeida, Stefanos Laskaridis, Ilias Leontiadis, Stylianos I. Venieris 和 Nicholas D. Lane. 2019. EmBench: 量化现代商品设备中深度神经网络的性能变化。第三届移动系统和应用深度学习国际研讨会 (EMDL)。ACM, 1-6. [2] Mario Almeida, Stefanos Laskaridis, Abhinav Mehrotra, Lukasz Dudziak, Ilias Leontiadis 和 Nicholas D. Lane. 2021. 智能的代价是什么? 描述野外移动深度神经网络的特征。第 21 届 ACM 互联网测量会议 (虚拟活动) (IMC '21) 会议记录。计算机协会, 美国纽约州纽约市, 658-672. <https://doi.org/10.1145/3487552.3487863> [3] Mattia Antonini, Tran Huy Vu, Chulhong Min, Alessandro Montanari, Akhil Mathur 和 Fahim Kawsar. 2019. 边缘加速器上个人规模传感模型的资源表征。首届物联网人工智能和机器学习挑战国际研讨会 (AIChallengeIoT) 会议记录。ACM, 49-55. [4] 戴维斯·布莱洛克、何塞·哈维尔·冈萨雷斯·奥尔蒂斯、乔纳森·弗兰克尔和约翰·古塔格。2020.神经网络剪枝的现状如何? 在《机器学习和系统学报》(MLSys) 中。卷。2. 129-146. [5] J.伯吉斯。2019年。RTX ON - NVIDIA TURING GPU。2019年IEEE Hot Chips 31研讨会 (HCS)。1-27. <https://doi.org/10.1109/HOTCHIPS.2019.8875651> [6] Alejandro Cartas, Martin Kocour, Aravindh Raman, Ilias Leontiadis, Jordi Luque, Nishanth Sastry, Jose Nuñez-Martinez, Diego Perino 和 Carlos Segura. 2019. 移动网络边缘推理的现实检验。第二届边缘系统、分析和网络国际研讨会 (EdgeSys) 论文集。54-59. [7] Byung-Gon Chun, Sunghwan Ihm, Petros Maniatis, Mayur Naik 和 Ashwin Patti. 2011. CloneCloud: 移动设备和云之间的弹性执行。第六届计算机系统会议 (EuroSys '11) 的会议记录。301-314. [8] E. Chung 等人。2018年。通过 Project Brainwave 在数据中心规模实时提供 DNN 服务。IEEE Micro 38, 2 (2018), 8-20. [9] Eduardo Cuervo, Aruna Balasubramanian, Dae-ki Cho, Alec Wolman, Stefan Saroiu, Ranveer Chandra 和 Paramvir Bahl. 2010. MAUI: 通过代码卸载让智能手机使用寿命更长。国际移动系统、应用程序和服务会议 (MobiSys)。
- [10] 拉胡尔·戴伊和法蒂·M·塞勒姆。2017。门控循环单元 (GRU) 神经网络的门变体。2017年IEEE第60届国际中西部电路与系统研讨会 (MWSCAS)。IEEE, 1597-1600.
- [11] 周放、林正浩、Mani B. Srivastava 和 Rajesh K. Gupta. 2019. 用于实时计算机视觉应用的多租户移动卸载系统。第 20 届国际分布式计算会议论文集

and Networking (ICDCN). 21–30.

- [12] Georgios Georgiadis. 2019. Accelerating Convolutional Neural Networks via Activation Map Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [13] Mark S. Gordon, D. Anoushe Jamshidi, Scott Mahlke, Z. Morley Mao, and Xu Chen. 2012. COMET: Code Offload by Migrating Execution Transparently. In *USENIX Conference on Operating Systems Design and Implementation (OSDI)*.
- [14] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, and H. Yang. 2018. Angel-Eye: A Complete Design Flow for Mapping CNN Onto Embedded FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 37, 1 (2018), 35–47.
- [15] P. Gysel, J. Pimentel, M. Motamedi, and S. Ghiasi. 2018. Ristretto: A Framework for Empirical Study of Resource-Efficient Inference in Convolutional Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)* 29, 11 (2018), 5784–5789.
- [16] Song Han, Huizi Mao, and William J Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*.
- [17] Seungyeop Han, Haichen Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman, and Arvind Krishnamurthy. 2016. MCDNN: An Approximation-Based Execution Framework for Deep Stream Processing Under Resource Constraints. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*.
- [18] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzhulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro, J. Law, K. Lee, J. Lu, P. Noordhuis, M. Smelyanskiy, L. Xiong, and X. Wang. 2018. Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 620–629.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *International Conference on Computer Vision (ICCV)*. 1026–1034.
- [20] K. He, X. Zhang, S. Ren, and J. Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.
- [21] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [22] Chuang Hu, Wei Bao, Dan Wang, and Fengming Liu. 2019. Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge. *Proceedings - IEEE INFOCOM* (2019), 1423–1431.
- [23] Jin Huang, Colin Samplawski, Deepak Ganesan, Benjamin Marlin, Heesung Kwon, and Paper Xxx. 2020. CLIO: Enabling Automatic Compilation of Deep Learning Pipelines across IoT and Cloud. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom)*.
- [24] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. 2019. AI Benchmark: All About Deep Learning on Smartphones in 2019. In *ICCV Workshops (ICCVW)*.
- [25] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [26] Nithilan Kanappan Jayakodi, Syrine Belakaria, Aryan Deshwal, and Janardhan Rao Doppa. 2020. Design and Optimization of Energy-Accuracy Tradeoff Networks for Mobile Platforms via Pretrained Deep Models. *ACM Trans. Embed. Comput. Syst. (TECS)* 19, 1, Article 4 (2020).
- [27] Nithilan Kannappan Jayakodi, Anwesha Chatterjee, Wonje Choi, Janardhan Rao Doppa, and Partha Pratim Pande. 2018. Trading-off Accuracy and Energy of Deep Inference on Embedded Systems: A Co-Design Approach. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 37, 11 (2018), 2881–2893.
- [28] Nithilan Kanappan Jayakodi, Janardhan Rao Doppa, and Partha Pratim Pande. 2020. SETGAN: Scale and Energy Trade-Off GANs for Image Applications on Mobile Platforms. In *Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD)*. 1–9.
- [29] Hyuk-Jin Jeong, Hyeon-Jae Lee, Chang Hyun Shin, and Soo-Mook Moon. 2018. IONN: Incremental Offloading of Neural Network Computations from Mobile Devices to Edge Servers. *Proceedings of the ACM Symposium on Cloud Computing* (2018), 401–411.
- [30] Norman P. Jouppi et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *International Symposium on Computer Architecture (ISCA)*.
- [31] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)* (2017), 615–629.
- [32] A. Kouris and C. Bouganis. 2018. Learning to Fly by MySelf: A Self-Supervised CNN-Based Approach for Autonomous Navigation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1–9.
- [33] A. Kouris, S. I. Venieris, and C. Bouganis. 2018. CascadeCNN: Pushing the Performance Limits of Quantisation in Convolutional Neural Networks. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. 155–1557.

和网络 (ICDCN)。21-30。[12] 乔吉奥斯·乔治亚迪斯。2019。通过激活图压缩加速卷积神经网络。IEEE/CVF 计算机视觉和模式识别会议 (CVPR) 会议记录。[13] Mark S. Gordon、D. Anoushe Jamshidi、Scott Mahlke、Z. Morley Mao 和 Xu Chen。2012.COMET: 通过透明地迁移执行来卸载代码。在 USENIX 操作系统设计与实现会议 (OSDI) 中。[14] 郭克, 隋丽, 邱建, 于建, 王建, 姚树, 韩树, 王勇, 杨红。2018. Angel-Eye: 将 CNN 映射到嵌入式 FPGA 的完整设计流程。IEEE 集成电路与系统计算机辅助设计 (TCAD) 37, 1 (2018), 35–47。[15] P. Gysel、J. Pimentel、M. Motamedi 和 S. Ghiasi。2018.Ristretto: 卷积神经网络中资源高效推理的实证研究框架。IEEE 神经网络和学习系统汇刊 (TNNLS) 29, 11 (2018), 5784–5789。[16] 韩松, 毛慧子, William J Dally。2016。深度压缩: 通过剪枝、训练量化和霍夫曼编码来压缩深度神经网络。在国际学习表征会议 (ICLR) 中。[17] Seungyeop Han、Haichen Shen、Matthai Philipose、Sharad Agarwal、Alec Wolman 和 Arvind Krishnamurthy。2016. MCDNN: 资源约束下基于近似的深度流处理执行框架。第 14 届移动系统、应用程序和服务国际会议 (MobiSys) 的会议记录。[18] K. Hazelwood、S. Bird、D. Brooks、S. Chintala、U. Diril、D. Dzhulgakov、M. Fawzy、B. Jia、Y. Jia、A. Kalro、J. Law、K. Lee、J. Lu、P. Noordhuis、M. Smelyanskiy、L. Xiong 和 X. Wang。2018 年。Facebook 的应用机器学习: 数据中心基础设施视角。2018 年 IEEE 高性能计算机架构国际研讨会 (HPCA)。620–629。[19] 何凯明, 张翔宇, 任少清, 孙健。2015 年。深入研究整流器: 在 ImageNet 分类上超越人类水平的性能。在国际计算机视觉会议 (ICCV) 上。1026–1034。[20] 何坤, 张旭, 任书, 孙静。2016. 用于图像识别的深度残差学习。在 IEEE 计算机视觉和模式识别会议 (CVPR) 中。770–778。[21] 塞普·霍克赖特 (Sepp Hochreiter) 和于尔根·施米德胡贝尔 (Jürgen Schmidhuber)。1997. 长短期记忆。神经计算 9, 8 (1997), 1735–1780。[22] 胡闯, 包伟, 王丹, 刘凤鸣。2019. 用于边缘推理加速的动态自适应 DNN 手术。会议记录 - IEEE INFOCOM (2019), 1423–1431。[23] Jin Huang、Colin Samplawski、Deepak Ganesan、Benjamin Marlin、Heesung Kwon 和 Paper Xxx。2020. CLIO: 实现跨物联网和云的深度学习管道的自动编译。第 26 届年度移动计算和网络国际会议 (MobiCom) 的会议记录。[24] Andrey Ignatov、Radu Timofte、Andrei Kulik、Seungsoo Yang、Ke Wang、Felix Baum、Max Wu、Lirong Xu 和 Luc Van Gool。2019. 人工智能基准: 2019 年智能手机深度学习的全部内容。在 ICCV 研讨会 (ICCVW) 中。[25] Benoit Jacob、Skirmantas Kligys、Bo Chen、Menglong Zhu、Matthew Tang、Andrew Howard、Hartwig Adam 和 Dmitry Kalenichenko。2018. 用于高效整数算术推理的神经网络的量化和训练。IEEE 计算机视觉和模式识别会议 (CVPR) 会议记录。[26] Nitthilan Kanappan Jayakodi、Syrine Belakaria、Aryan Deshwal 和 Janardhan Rao Doppa。2020. 通过预训练深度模型设计和优化移动平台的能量精度权衡网络。ACM 翻译。嵌入。计算。系统。(TECS) 19, 1, 第 4 条 (2020)。[27] Nitthilan Kannappan Jayakodi、Anwesha Chatterjee、Wonje Choi、Janardhan Rao Doppa 和 Partha Pratim Pandey。2018. 权衡嵌入式系统深度推理的准确性和能量: 一种协同设计方法。IEEE 集成电路与系统计算机辅助设计 (TCAD) 37, 11 (2018), 2881–2893。[28] Nitthilan Kanappan Jayakodi、Janardhan Rao Doppa 和 Partha Pratim Pande。2020. SETGAN: 移动平台上图像应用的规模和能源权衡 GAN。第 39 届国际计算机辅助设计会议 (ICCAD) 论文集。1-9。[29] Hyuk-Jin Jeong、Hyeon-Jae Lee、Chang Hyun Shin 和 Soo-Mook Moon。2018. IONN: 将神经网络计算从移动设备增量卸载到边缘服务器。ACM 云计算研讨会论文集 (2018 年), 401-411。[30] 诺曼·P·乔皮 (Norman P. Jouppi) 等人。2017. 张量处理单元的数据中心内性能分析。在国际计算机体系结构研讨会 (ISCA) 中。[31] 康一平、约翰·豪斯瓦尔德、曹高、奥斯汀·罗文斯基、特雷弗·马奇、杰森·马尔斯和唐凌嘉。2017. 神经外科医生: 云和移动边缘之间的协作智能。编程语言和操作系统架构支持国际会议 (ASPLOS) (2017), 615–629

- [34] Alexandros Kouris, Stylianos I. Venieris, Stefanos Laskaridis, and Nicholas D. Lane. 2021. Multi-Exit Semantic Segmentation Networks. In *arXiv*.
- [35] V. K. Kukkala, J. Tunnell, S. Pasricha, and T. Bradley. 2018. Advanced Driver-Assistance Systems: A Path Toward Autonomous Vehicles. *IEEE Consumer Electronics Magazine* 7, 5 (2018), 18–25.
- [36] Stefanos Laskaridis, Alexandros Kouris, and Nicholas D. Lane. 2021. Adaptive Inference through Early-Exit Networks: Design, Challenges and Directions. In *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning (EMDL)*. 1–6.
- [37] Stefanos Laskaridis, Stylianos I. Venieris, Mario Almeida, Ilias Leontiadis, and Nicholas D. Lane. 2020. SPINN: Synergistic Progressive Inference of Neural Networks over Device and Cloud. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom)*.
- [38] Stefanos Laskaridis, Stylianos I. Venieris, Hyeji Kim, and Nicholas D. Lane. 2020. HAPI: Hardware-Aware Progressive Inference. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9.
- [39] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and June-hwa Song. 2019. Occlumency: Privacy-Preserving Remote Deep-Learning Inference Using SGX. In *The 25th Annual International Conference on Mobile Computing and Networking (MobiCom)*.
- [40] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. 2019. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications (TWC)* 19, 1 (2019), 447–457.
- [41] Hongshan Li, Chenghao Hu, Jingyan Jiang, Zhi Wang, Yonggang Wen, and Wenwu Zhu. 2019. JALAD: Joint Accuracy-And Latency-Aware Deep Structure Decoupling for Edge-Cloud Execution. In *International Conference on Parallel and Distributed Systems (ICPADS)*. 671–678.
- [42] Jiachen Mao, Xiang Chen, Kent W. Nixon, Christopher Krieger, and Yiran Chen. 2017. MoDNN: Local distributed mobile computing system for Deep Neural Network. In *Design, Automation and Test in Europe (DATE)*.
- [43] Jiachen Mao, Zhongda Yang, Wei Wen, Chunpeng Wu, Linghao Song, Kent W. Nixon, Xiang Chen, Hai Li, and Yiran Chen. 2017. MeDNN: A distributed mobile system with enhanced partition and deployment for large-scale DNNs. In *International Conference on Computer-Aided Design (ICCAD)*. 751–756.
- [44] Kiyoshi Masui, Mandana Amiri, Liam Connor, Meiling Deng, Mateus Fandino, Carolin Höfer, Mark Halpern, David Hanna, Adam D Hincks, Gary Hinshaw, et al. 2015. A compression scheme for radio data in high performance computing. *Astronomy and Computing* 12 (2015), 181–190.
- [45] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. DarknetZ: Towards Model Privacy at the Edge Using Trusted Execution Environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services (MobiSys)*. 161–174.
- [46] Vinod Nair and Geoffrey E. Hinton. 2010. Rectified Linear Units Improve Restricted Boltzmann Machines. In *International Conference on Machine Learning (ICML)*. 807–814.
- [47] M. Nikolić, M. Mahmoud, and A. Moshovos. 2018. Characterizing Sources of Ineffectual Computations in Deep Learning Networks. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*. 86–87.
- [48] Ofcom. 2014. 3G and 4G Network Speeds. <https://www.ofcom.org.uk/about-ofcom/latest/media/media-releases/2014/3g-4g-bb-speeds>.
- [49] Priyadarshini Panda, Abhronil Sengupta, and Kaushik Roy. 2016. Conditional Deep Learning for Energy-Efficient and Enhanced Pattern Recognition. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 475–480.
- [50] PyTorch. 2021. Dynamic Quantization. https://pytorch.org/tutorials/recipes/recipes/dynamic_quantization.html. [Retrieved: January 12, 2022].
- [51] Prajit Ramachandran, Barret Zoph, and Quoc V Le. 2018. Searching for Activation Functions. In *International Conference on Learning Representations (ICLR) workshops*.
- [52] M. Rhu, M. O'Connor, N. Chatterjee, J. Pool, Y. Kwon, and S. W. Keckler. 2018. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In *International Symposium on High Performance Computer Architecture (HPCA)*. 78–91.
- [53] Mark D. Ryan. 2011. Cloud Computing Privacy Concerns on Our Doorstep. *Commun. ACM* 54, 1 (2011), 36–38.
- [54] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4510–4520.
- [55] K Simonyan and A Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations (ICLR)*.
- [56] V. Sze, Y. Chen, T. Yang, and J. S. Emer. 2017. Efficient Processing of Deep Neural Networks: A Tutorial and Survey. *Proc. IEEE* 105, 12 (2017), 2295–2329.
- [57] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander Alemi. 2017. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. In *AAAI Conference on Artificial Intelligence*.

[34] Alexandros Kouris, Stylianos I. Venieris, Stefanos Laskaridis 和 Nicholas D. Lane. 2021.多出口语义分割网络。在 arXiv 中。[35] V.K. Kukkala, J. Tunnell, S. Pasricha 和 T. Bradley. 2018. 高级驾驶辅助系统: 迈向自动驾驶汽车的道路。IEEE 消费电子杂志 7, 5 (2018), 18–25. [36] Stefanos Laskaridis, Alexandros Kouris 和 Nicholas D. Lane. 2021. 通过早期退出网络进行自适应推理: 设计、挑战 and 方向。第五届嵌入式和移动深度学习国际研讨会 (EMDL) 论文集。1–6. [37] Stefanos Laskaridis, Stylianos I. Venieris, Mario Almeida, Ilias Leontiadis 和 Nicholas D. Lane. 2020.SPINN: 设备和云上神经网络的协同渐进推理。第 26 届年度移动计算和网络国际会议 (MobiCom) 的会议记录。[38] Stefanos Laskaridis, Stylianos I. Venieris, Hyeji Kim 和 Nicholas D. Lane. 2020.HAPI: 硬件感知渐进推理。2020 年 IEEE/ACM 计算机辅助设计国际会议 (ICCAD)。1–9. [39] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, Junehwa Song. 2019. 大脑封闭术: 使用 SGX 保护隐私的远程深度学习推理。第 25 届年度移动计算和网络国际会议 (MobiCom)。[40] 李恩, 曾列康, 周志, 陈旭. 2019. 边缘人工智能: 通过边缘计算按需加速深度神经网络推理。IEEE 无线通信汇刊 (TWC) 19, 1 (2019), 447–457. [41] 李洪山, 胡成浩, 蒋敬艳, 王志, 文永刚, 朱文武. 2019.JALAD: 边缘云执行的联合精度和延迟感知深度结构解耦。在国际并行和分布式系统会议 (ICPADS) 中。671–678. [42] 毛嘉辰, 陈翔, 肯特·W·尼克松, 克里斯托弗·克里格, 陈怡然. 2017. MoDNN: 深度神经网络的本地分布式移动计算系统。在欧洲的设计、自动化和测试 (日期)。[43] 毛佳辰, 杨忠达, 文伟, 吴春鹏, 宋凌浩, 肯特·尼克松, 陈翔, 李海, 陈怡然. 2017.MeDNN: 一种分布式移动系统, 具有针对大规模 DNN 的增强分区和部署。在国际计算机辅助设计会议 (ICCAD) 中。751–756. [44] Kiyoshi Masui, Mandana Amiri, Liam Connor, Meiling Deng, Mateus Fandino, Carolin Höfer, Mark Halpern, David Hanna, Adam D Hincks, Gary Hinshaw 等. 2015.高性能计算中无线电数据的压缩方案。天文学与计算 12 (2015), 181–190. [45] 范莫, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro 和 Hamed Haddadi. 2020.DarkneTZ: 使用可信执行环境实现边缘隐私模型。第 18 届国际移动系统、应用程序和服务会议 (MobiSys) 论文集。161–174. [46] 维诺德·奈尔和杰弗里·E·辛顿. 2010. 整流线性单元改进了受限玻尔兹曼机。在国际机器学习会议 (ICML) 中。807–814. [47] M. Nikolić, M. Mahmoud 和 A. Moshovos. 2018. 描述深度学习网络中无效计算的来源。2018 年 IEEE 工作负载表征国际研讨会 (IISWC)。86–87. [48] 英国通信管理局. 2014 年. 3G 和 4G 网络速度。https://www.ofcom.org.uk/about-ofcom/latest/media/media-releases/2014/3g-4g-bb-speeds. [49] Priyadarshini Panda, Abhronil Sengupta 和 Kaushik Roy. 2016. 用于节能和增强模式识别的条件深度学习。2016 年欧洲设计、自动化和测试会议暨展览 (日期)。IEEE, 475–480. [50] 火炬. 2021. 动态量化。https://pytorch.org/tutorials/recipes/recipes/dynamic_quantization.html. [检索日期: 2022 年 1 月 12 日]。[51] Prajit Ramachandran, Barret Zoph 和 Quoc V Le. 2018. 寻找激活函数。在国际学习表征会议 (ICLR) 研讨会上。[52] M. Rhu, M. O’ Connor, N. Chatterjee, J. Pool, Y. Kwon 和 S. W. Keckler. 2018. 压缩 DMA 引擎: 利用激活稀疏性训练深度神经网络。在高性能计算机体系结构 (HPCA) 国际研讨会上。78–91. [53] 马克·D·瑞安. 2011 年. 我们家门口的云计算隐私问题。交流。ACM 54, 1 (2011), 36–38. [54] 马克·桑德勒, 安德鲁·霍华德, 朱梦龙, 安德烈·兹莫吉诺夫和陈良杰. 2018. MobileNetV2: 倒置残差和线性瓶颈。在 IEEE 计算机视觉和模式识别会议 (CVPR) 中。4510–4520. [55] K Simonyan 和 A Zisserman. 2015. 用于大规模图像识别的超深卷积网络。在国际学习表征会议 (ICLR) 中。[56] V. Sze, Y. Chen, T. Yang 和 J. S. Emer. 2017. 神经网络的高效处理: 教程和调查。过程。IEEE 105, 12 (2017), 2295

- [58] Asoke K Talukder, Lawrence Zimmerman, et al. 2010. Cloud economics: Principles, costs, and benefits. In *Cloud computing*. Springer, 343–360.
- [59] C. Wu, D. Brooks, K. Chen, D. Chen, S. Choudhury, M. Dukhan, K. Hazelwood, E. Isaac, Y. Jia, B. Jia, T. Leyvand, H. Lu, Y. Lu, L. Qiao, B. Reagen, J. Spisak, F. Sun, A. Tulloch, P. Vajda, X. Wang, Y. Wang, B. Wasti, Y. Wu, R. Xian, S. Yoo, and P. Zhang. 2019. Machine Learning at Facebook: Understanding Inference at the Edge. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*.
- [60] Xiaowei Xu, Yukun Ding, Sharon Xiaobo Hu, Michael Niemier, Jason Cong, Yu Hu, and Yiyu Shi. 2018. Scaling for edge inference of deep neural networks. *Nature Electronics* 1 (04 2018).
- [61] Shuochao Yao, Shaohan Hu, Yiran Zhao, Aston Zhang, and Tarek Abdelzaher. 2017. DeepSense: A Unified Deep Learning Framework for Time-Series Mobile Sensing Data Processing. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*. 351–360.
- [62] Yuan Zhang, Hao Liu, Lei Jiao, and Xiaoming Fu. 2012. To offload or not to offload: An efficient code partition algorithm for mobile cloud computing. In *2012 IEEE 1st International Conference on Cloud Networking (CLOUDNET)*. 80–86.
- [63] Minjia Zhang, Samyam Rajbhandari, Wenhan Wang, and Yuxiong He. 2018. DeepCPU: Serving RNN-based Deep Learning Models 10x Faster. In *2018 USENIX Annual Technical Conference*. 951–965.
- [64] Wuyang Zhang, Zhezhi He, Luyang Liu, Zhenhua Jia, Yunxin Liu, Marco Gruteser, Dipankar Raychaudhuri, and Yanyong Zhang. 2021. Elf: Accelerate High-Resolution Mobile Deep Vision with Content-Aware Parallel Offloading. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (MobiCom)*. Association for Computing Machinery, New York, NY, USA, 201–214. <https://doi.org/10.1145/3447993.3448628>
- [65] Zhuoran Zhao, Kamyar Mirzazad Barijough, and Andreas Gerstlauer. 2018. DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 37, 11 (2018).
- [66] Aojun Zhou et al. 2017. Incremental Network Quantization: Towards Lossless CNNs with Low-Precision Weights. In *International Conference on Learning Representations (ICLR)*.

[58] Asoke K Talukder, Lawrence Zimmerman 等。2010。云经济学：原理、成本和收益。在云计算中。施普林格，343-360。[59] C. Wu, D. Brooks, K. Chen, D. Chen, S. Choudhury, M. Dukhan, K. Hazelwood, E. Isaac, Y. Jia, B. Jia, T. Leyvand, H. Lu, Y. Lu, L. Qiao, B. Reagen, J. Spisak, F. Sun, A. Tulloch, P. Vajda, X. Wang, Y. Wang, B. Wasti, Y. 吴, R. 西安, S. Yoo 和 P. 张。2019。Facebook 的机器学习：理解边缘推理。在 IEEE 国际高性能计算机体系结构 (HPCA) 研讨会上。[60] 徐晓伟, 丁玉坤, 胡晓波, 迈克尔·尼米尔, 丛杰森, 胡宇, 施一字。2018。深度神经网络边缘推理的扩展。自然电子学 1 (2018 年 4 月)。[61] 姚硕超, 胡少涵, 赵怡然, 张阿斯頓, 塔里克·阿卜杜勒扎赫。2017。DeepSense：用于时间序列移动传感数据处理的统一深度学习框架。在第 26 届万维网国际会议 (WWW) 的会议记录中。351-360。[62] 张元, 刘浩, 焦雷, 付晓明。2012。卸载还是不卸载：移动云计算的高效代码划分算法。2012 年 IEEE 第一届云网络国际会议 (CLOUDNET)。80-86。[63] 张敏佳, Samyam Rajbhandari, 王文瀚, 何宇雄。2018。DeepCPU：服务基于 RNN 的深度学习模型速度提高 10 倍。2018 年 USENIX 年度技术会议。951-965。[64] 张武阳, 何哲智, 刘路阳, 贾振华, 刘云欣, Marco Gruteser, Dipankar Raychaudhuri, 张彦勇。2021。Elf：通过内容感知并行卸载加速高分辨率移动深度视觉。第 27 届移动计算和网络年度国际会议 (MobiCom) 的会议记录。计算机协会, 美国纽约州纽约市, 201-214。https://doi.org/10.1145/3447993.3448628 [65] 赵卓然, Kamyar Mirzazad Barijough 和 Andreas Gerstlauer。2018。DeepThings：资源受限的物联网边缘集群上的分布式自适应深度学习推理。IEEE 集成电路和系统计算机辅助设计 (TCAD) 交易 37, 11 (2018)。[66] 周傲君等。2017。增量网络量化：迈向低精度权重的无损 CNN。在国际学习表征会议 (ICLR) 中。